# Package 'rpbnb'

September 6, 2026

**Type** Package

**Title** Random-Parameter Bivariate Negative Binomial Regression

**Version** 0.4.6

**Author** Zhenyu Wang [aut, cre]

**Maintainer** Zhenyu Wang <wonstran@hotmail.com>

**Description** Maximum-likelihood estimation of bivariate negative binomial (NB2) regression models with Famoye/Sarmanov or discrete-copula (Frank, Gaussian, or Clayton) dependence, and maximum-simulated-likelihood estimation of a bivariate random-parameter negative binomial model under either dependence structure, with a random-coefficient data simulator, diagnostics, and standard model methods. Two estimation engines are provided: an OpenMP/Rcpp maximum-simulated-likelihood engine and a TMB automatic-differentiation engine that additionally offers a Laplace approximation, with a common front end.

**License** MIT + file LICENSE

**Encoding** UTF-8

**Depends** R (>= 4.1)

**Imports** stats,
compiler,
maxLik,
numDeriv,
randtoolbox,
MASS,
pbivnorm,
Rcpp,
parallel,
graphics,
TMB

**LinkingTo** Rcpp,
TMB,
RcppEigen

**Suggests** testthat (>= 3.1.5),
knitr,
rmarkdown,
pkgload

**SystemRequirements** GNU make

**Config/testthat/edition** 3

**VignetteBuilder** knitr

**Roxygen** list(markdown = TRUE)

**LazyData** false

**Config/roxygen2/version** 8.0.0

# Contents

bnb_boundary_tests *Boundary-corrected LR tests for the NB2 dispersions of a fit_bnb() fit*

## Description

Runs a likelihood-ratio test for each free NB2 dispersion (m1, m2) of a fitted fixed-coefficient bivariate NB model, against a restricted refit at that margin's exact Poisson limit (m = 0). m = 0 sits on the boundary of the parameter space, so the natural-scale summary reports no Wald z/p for m1/m2 (see fit_bnb()); the valid test is lr_test()'s boundary = TRUE 50:50 chi-square mixture. This is the fixed-coefficient counterpart of rpbnb_boundary_tests() – the same test, restricted to the one boundary-null group fit_bnb() has (no random-coefficient scales).

## Usage

```
bnb_boundary_tests(fit, data, control = rpbnb_control(compute_se = FALSE))
```

## Arguments

| | |
|---|---|
| fit | A converged bnb_fit (from fit_bnb()) with dependence = "independence" or "famoye". Not supported for a copula() dependence: fit_bnb() itself refuses to combine poisson_1/poisson_2 with a copula dependence, so there is no Poisson-limit restriction to test. |
| data | The data frame the model was fit on. Required – the fit object does not store it, and every restricted model is refit on it. |
| control | An rpbnb_control() for the restricted refits. Defaults to compute_se = FALSE (the LR test needs only logLik and the degrees of freedom). |

## Details

Each restricted refit is warm-started from the full fit's own coefficients (start = fit$coef); fit_bnb() pins the tested margin's log_m to the Poisson placeholder regardless of what start supplies for it, so passing the full vector is safe and needs no per-test trimming. Unlike rpbnb_boundary_tests(), there is no warm-start "polish" step for a negative LR statistic: fit_bnb()'s likelihood is exact (glm.nb for independence, closed-form-gradient BFGS for Famoye), not simulated, so a warm-started restricted refit does not out-climb the full fit's own optimum the way a simulated objective occasionally can.

## Value

An object of class bnb_boundary_tests (a data frame with columns Parameter, LR, df, p.value, Signif, one row per tested margin) and a print method. A margin already fit at poisson_1/poisson_2 = TRUE has no free dispersion left to test and is skipped, matching rpbnb_boundary_tests()'s dispersion-test behaviour.

## See Also

lr_test(), fit_bnb(), rpbnb_boundary_tests()

## Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_boundary_tests(fit, d)
```

---

`bnb_elasticities` *Elasticities and semi-elasticities for a bivariate NB model*

---

## Description

For continuous regressors, the elasticity is $\beta_j E[x_j]$; for binary (0/1) regressors, the semi-elasticity is $\exp(\beta_j) - 1$ (proportional change moving from 0 to 1).

## Usage

```
bnb_elasticities(
  fit,
  which = c("y1", "y2", "both"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE
)
```

## Arguments

| | |
|---|---|
| `fit` | A `bnb_fit` object from `fit_bnb()`. |
| `which` | Which margin(s): "y1", "y2", or "both". |
| `type` | "AME" (uses sample mean of x) or "MEM" (uses mean design row). |
| `vars` | Optional variable names to restrict output. |
| `include_intercept` | Logical; include the intercept term. |
| `digits` | Number of decimal places for printed output. |
| `print_output` | Logical; if `FALSE`, suppress printing. |

## Value

A data frame (single margin) or a named list of data frames (both), each with columns `Name`, `Estimate`, `StdErr`, `z`, `p`, `Signif`, `var_type`.

## Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork + age, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_elasticities(fit, which = "both", type = "AME")
```

## bnb_gof *Goodness of fit for a bivariate NB model*

### Description

Reports log-likelihood, AIC, BIC, and four pseudo-R-squared measures (McFadden, McFadden adjusted, Cox-Snell, Nagelkerke) relative to an intercept-only null model refit with the same dependence structure (copula fits rebuild the copula from `fit$cop_family`). A margin fitted with an `offset()` keeps that offset in the null (an intercept-plus-offset model), so the null shares the full model's exposure/sampling structure and the pseudo-R-squared values remain comparable. Pseudo-R-squared values are returned raw and are not clamped to `[0, 1]`; a negative value flags a full model that fits worse than the null.

### Usage

```
bnb_gof(fit, digits = 4, print_output = TRUE)
```

### Arguments

| | |
|---|---|
| `fit` | A `bnb_fit` object from `fit_bnb()`. |
| `digits` | Number of decimal places for printed output. |
| `print_output` | Logical; if `FALSE`, suppress printing and return the result invisibly. |

### Value

Invisibly, a list with `n`, `k`, `logLik_full`, `logLik_null`, `AIC`, `BIC`, a named numeric vector `pseudoR2`, and the `null_fit`.

### Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork + age, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_gof(fit)
```

## bnb_marginal_effects *Marginal effects for a bivariate NB model*

### Description

Average marginal effects (AME) or marginal effects at the mean (MEM) for each margin. Continuous and binary (0/1) regressors are auto-detected; continuous effects use $\partial E[Y]/\partial x_j = \beta_j \mu$ and binary effects use $E[Y|x_j = 1] - E[Y|x_j = 0]$.

### Usage

```
bnb_marginal_effects(
  fit,
  which = c("y1", "y2", "both", "all"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE
)
```

### Arguments

| | |
|---|---|
| `fit` | A `bnb_fit` object from `fit_bnb()`. |
| `which` | Which margin(s): "y1", "y2", "both", or "all". |
| `type` | "AME" (average marginal effect) or "MEM" (effect at the mean). |
| `vars` | Optional variable names or indices to restrict output. |
| `include_intercept` | Logical; include the intercept term. |
| `digits` | Number of decimal places for printed output. |
| `print_output` | Logical; if `FALSE`, suppress printing. |

### Details

For a model fitted with an `offset()`, absolute marginal effects scale with the fitted mean $\mu = \exp(x'\beta + \text{offset})$: AME uses each observation's training offset, and MEM uses the mean training offset.

### Value

A data frame (single margin) or a named list of data frames (both), each with columns `Name`, `Estimate`, `StdErr`, `z`, `p`, `Signif`, `var_type`.

### Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork + age, hospvis ~ outwork, data = d,
               dependence = "famoye")
bnb_marginal_effects(fit, which = "y1", type = "AME")
```

---

## `bnb_residual_checks` *Residual checks for a bivariate NB model*

---

### Description

Formal residual diagnostics for a `fit_bnb()` or `fit_rpbnb()` fit: a normality test on the randomized quantile residuals (Shapiro-Wilk for `n <= 5000`, else Kolmogorov-Smirnov vs N(0,1)); the NB2 dispersion statistic (`sum(pearson^2) / (n - k)`) per margin; the cross-margin RQR correlation (a check that the Famoye/copula dependence captured the association); an outlier count/index list (`|RQR| > outlier_z`); and a composite misspecification verdict combining these signals.

### Usage

```
bnb_residual_checks(
  fit,
  seed = NULL,
  outlier_z = 3,
  digits = 4,
  print_output = TRUE
)
```

### Arguments

| | |
|---|---|
| fit | A bnb_fit or rpbnb_fit object. |
| seed | Optional integer seed for the RQR randomization. |
| outlier_z | Absolute-RQR threshold flagging an outlier (default 3). |
| digits | Decimal places for printed output. |
| print_output | Logical; if FALSE, suppress printing. |

### Value

Invisibly, an object of class bnb_residual_checks.

---

## copula *Specify a copula dependence structure*

---

### Description

Pass the result as the dependence argument to fit_bnb() or fit_rpbnb() to estimate a discrete-copula bivariate NB model instead of the default Famoye/Sarmanov dependence, or as the copula argument to simulate_rpbnb_copula() to simulate from one. The joint pmf is built from the two NB2 marginal CDFs and the chosen copula CDF (rectangle differencing); see the package vignette for the "Copula dependence" section.

### Usage

```
copula(family = c("frank", "normal", "kimeldorf"), par = NULL)
```

### Arguments

| | |
|---|---|
| family | One of "frank", "normal" (Gaussian), or "kimeldorf" (Clayton). |
| par | Optional native dependence parameter (Frank's theta, the Gaussian rho, or Clayton's theta) used by simulate_rpbnb_copula() to generate data. Ignored by fit_bnb() and fit_rpbnb(), which estimate the parameter from the data. |

### Value

An object of class rpbnb_copula.

### Examples

```
copula("frank")
copula("normal", par = 0.3)
copula("kimeldorf")
```

`fit_bnb` *Fit a bivariate negative binomial regression model*

## Description

Fit a bivariate negative binomial regression model

## Usage

```
fit_bnb(
  formula_1,
  formula_2,
  data,
  dependence = c("independence", "famoye"),
  start = NULL,
  control = rpbnb_control(),
  poisson_1 = FALSE,
  poisson_2 = FALSE,
  boundary_tests = FALSE
)
```

## Arguments

`formula_1`, `formula_2`
: Model formulas for the two count outcomes. An equation-specific `offset()` term (e.g. `y ~ x + offset(log(exposure))`) is supported on every dependence path: the offset enters that margin's linear predictor additively (mean `exp(x'beta + offset)`) during estimation, and is carried through the stored fitted means and both `predict()` methods.

`data`
: A data frame.

`dependence`
: Dependence structure: "independence" (two univariate NB2 margins), "famoye" (Famoye/Sarmanov bivariate NB), or a `copula()` object (Frank / Gaussian / Clayton discrete-copula bivariate NB; the dependence parameter is estimated).

`start`
: Optional starting parameter vector. May be positional (length equal to the number of parameters) or named; a named vector is reordered to the canonical parameter order and a named partial vector is merged into the defaults (unknown or duplicate names are rejected). When `start` is `NULL`, the famoye path uses a multi-start policy: it optimizes from both an all-zero mean-coefficient start and marginal `glm.nb` starts and keeps the better converged objective (the frozen-bounds gradient makes the objective start-sensitive and neither start dominates).

`control`
: An `rpbnb_control()` object. The famoye and copula estimators both use BFGS, the only optimizer `control$method` accepts. One control object serves every estimator in the package; settings this one does not read – `se_method`, `n_cores`, `halton_burn`, `draws_hessian`, and the TMB knobs – are ignored and listed by `print()`/`summary()` of the fit.

`poisson_1`, `poisson_2`
: Fit the corresponding margin at its exact Poisson limit (NB2 dispersion `m = 0`) instead of estimating the dispersion. The margin's `log_m` is held fixed, so it is not a free parameter and the fit is a properly nested restriction of the NB model – pair it with `lr_test()` (`boundary = TRUE`) to test for overdispersion. This is the

exact `m = 0` restriction at any fitted mean: the margin's log-pmf is `dpois` and its Famoye dependence constant is `exp(-d*mu)` (the `m -> 0` limit), not an NB2 at a tiny pinned dispersion. The famoye and independence paths are both exact (the independence path fits a Poisson GLM margin). Not supported with a `copula()` dependence.

`boundary_tests` Run `bnb_boundary_tests()` on the converged fit and attach the result as `fit$boundary_tests`, so `print()`/`summary()` fill in the `m1`/`m2` LR/df/p columns instead of leaving them blank (see `bnb_boundary_tests()` for what the test is and why a Wald z/p is not valid there). Default `FALSE`: this costs up to two extra refits (one per free margin), so it is opt-in rather than automatic. Not supported with a `copula()` dependence, for the same reason `poisson_1`/`poisson_2` are not.

## Value

An object of class `bnb_fit`.

## Examples

```
d <- read.csv(system.file("extdata", "rwm1984_clean.csv", package = "rpbnb"))
fit <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
               dependence = "famoye")
summary(fit)

# Overdispersion test for margin 1 (H0: m1 = 0, Poisson)
fit_p1 <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
                  dependence = "famoye", poisson_1 = TRUE)
lr_test(fit_p1, fit, boundary = TRUE)

# Same test for both margins, run automatically and attached to the fit
fit2 <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
                dependence = "famoye", boundary_tests = TRUE)
fit2$boundary_tests

# Gaussian copula dependence instead of Famoye/Sarmanov
fit_cop <- fit_bnb(docvis ~ outwork, hospvis ~ outwork, data = d,
                   dependence = copula("normal"))
fit_cop$cop_tau  # estimated Kendall's tau
```

---

# `fit_rpbnb` *Fit a bivariate random-parameter negative binomial model*

---

## Description

Maximum simulated likelihood estimation with normal random coefficients and Famoye/Sarmanov dependence. Random coefficients are selected per equation by name via `random_1` / `random_2`.

## Usage

```
fit_rpbnb(
  formula_1,
  formula_2,
  data,
  random_1 = NULL,
```

```
  random_2 = NULL,
  draws = 400,
  draw_type = "halton",
  seed = 1234,
  start = NULL,
  control = rpbnb_control(),
  dependence = "famoye",
  poisson_1 = FALSE,
  poisson_2 = FALSE,
  .fixed = NULL,
  .opt_draws = NULL
)
```

## Arguments

| | |
|---|---|
| formula_1, formula_2 | Model formulas for the two count outcomes. An equation-specific offset() term (e.g. y ~ x + offset(log(exposure))) is supported: the offset enters that margin's linear predictor additively (integrated mean E[exp(x'beta + offset)]) during estimation and is carried through the stored fitted means and predict(). |
| data | A data frame. |
| random_1, random_2 | Random coefficients per equation. Either a character vector of model.matrix column names (all Normal), or a named list whose values are a distribution name ("normal", "lognormal", "uniform", "triangular") or a list list(dist = ..., sign = ...) (sign is -1/1 and lognormal-only). NULL means all-fixed for that equation. |
| draws | Number of simulation draws for the optimization. |
| draw_type | Quasi-random draw type. Only "halton" is supported in this version. |
| seed | Random seed for the simulation draws. |
| start | Optional starting parameter vector. |
| control | An rpbnb_control() object. Estimation uses BFGS, the only optimizer control$method accepts. One control object serves every estimator in the package; settings this one does not read – the TMB knobs (gradtol, restarts, max_threads, max_workload, parallel_tape), hessian, and draws_hessian – are ignored and listed by print()/summary() of the fit rather than rejected. |
| dependence | Dependence structure: "famoye" (default; Famoye/Sarmanov) or an copula() object for copula dependence (Frank / Gaussian / Clayton). Both paths use the multithreaded (OpenMP) C++ simulated likelihood; the copula path is more numerically expensive per evaluation (discrete-copula pmf + per-draw NB CDF corners), so fits typically take noticeably longer than the Famoye path at comparable draws/n. Random coefficients on 0/1 dummy regressors are weakly identified under the copula path (NB dispersion trades off against the random-coefficient scale); prefer random coefficients on continuous regressors. |
| poisson_1, poisson_2 | Fit the corresponding margin at its exact Poisson limit (NB2 dispersion m = 0): the margin's log_m is held fixed, so it is not a free parameter and the fit is a properly nested restriction of the NB model. Pair with lr_test() (boundary = TRUE) to test a margin for overdispersion (H0: m = 0). The simulated likelihood uses the exact m = 0 branch – the per-draw margin log-pmf is dpois and its Famoye dependence constant is exp(-d*mu) – so it is accurate at any fitted |

mean, not a fixed-dispersion approximation. Supported with both Famoye/Sarmanov and `copula()` dependence (the copula path uses the same exact m = 0 branch).

`.fixed` Internal. A named numeric vector of parameters (in the optimization/log-scale parameterization) to pin at the supplied values and hold fixed during estimation. Used by `rpbnb_boundary_tests()` to construct scale-zero (SD boundary) restricted fits; not intended for direct use.

`.opt_draws` Internal. A list `list(Z1, Z2)` of uniform Halton draw matrices to use verbatim instead of generating them, so a restricted refit reuses a full fit's draws (common random numbers). Used by `rpbnb_boundary_tests()`; not intended for direct use.

## Value

An object of class `rpbnb_fit`. Under Famoye dependence three fields describe the admissible lambda interval: `bounds` is the interval the optimized likelihood actually used, frozen at the starting values (its width also feeds `summary()`'s delta-method standard error for lambda); `bounds_at_optimum` is the interval admissible at the fitted parameters, recomputed after optimization for validation only; and `lambda_admissible` records whether `lambda` — the fitted value, mapped through `bounds` — lies inside `bounds_at_optimum`. `FALSE` means the optimizer escaped the valid region (a warning is raised) and the fit should be re-run from starting values closer to the optimum. For normal or lognormal random coefficients loaded in both margins the bound is the constant `c(-1, 1)`, the two intervals coincide, and escape cannot occur.

## Examples

```
sim <- simulate_rpbnb(n = 600,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 100, control = rpbnb_control(compute_se = FALSE))
coef(fit)


# Copula dependence instead of Famoye/Sarmanov (slower; fewer draws here
# for a quick example -- use more in practice)
sim_cop <- simulate_rpbnb_copula(n = 600,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.3)),
  dispersion = c(m1 = 0.4, m2 = 0.5),
  copula = copula("normal", par = 0.5), seed = 1)
fit_cop <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim_cop$data, random_1 = "x1",
                     dependence = copula("normal"), draws = 100,
                     control = rpbnb_control(compute_se = FALSE))
tanh(coef(fit_cop)[["z_theta"]])  # estimated copula rho
```

---

`fit_rpbnb_tmb` *Fit a bivariate random-parameter negative binomial model (TMB)*

---

## Description

Maximum simulated likelihood via TMB (Template Model Builder) with automatic differentiation. Supports Famoye/Sarmanov and discrete-copula (Frank, Gaussian, Clayton) dependence.

## Usage

```
fit_rpbnb_tmb(
  formula_1,
  formula_2,
  data,
  random_1 = NULL,
  random_2 = NULL,
  draws = 400L,
  seed = 1234L,
  start = NULL,
  dependence = "famoye",
  control = rpbnb_tmb_control(),
  inference = c("full", "diag", "none"),
  keep = c("postfit", "compact", "full"),
  poisson_1 = FALSE,
  poisson_2 = FALSE,
  method = c("sml", "laplace"),
  disable_parallel_gaussian = FALSE,
  force_parallel_gaussian = NULL,
  .fixed = NULL
)
```

## Arguments

| | |
|---|---|
| formula_1, formula_2 | Model formulas for the two count outcomes. |
| data | A data frame. |
| random_1, random_2 | Random coefficient specs per equation. NULL (all fixed), character vector (all Normal), or named list with per-variable distribution specs ("normal", "lognormal", "uniform", "triangular"). |
| draws | Number of Halton simulation draws. Under method = "sml" this sets the simulation grid the likelihood is averaged over. Tape size scales with nrow(data) * draws unless the fit draw-chunks (see control$tape_chunks at rpbnb_control()): when the weighted workload exceeds control$max_workload, the fit is split into several equal-sized draw chunks replayed over one smaller TMB tape, dropping peak memory to nrow(data) * ceiling(draws / chunks) at the cost of somewhat slower gradient evaluations – exact for the requested draws, not an approximation. Under method = "laplace" draws does not affect the likelihood, the tape, or chunking; its only remaining job is sizing the Halton grid used for the post-estimation averaging in predict() and the marginal-effect functions. (The frozen Famoye admissible-lambda interval is derived from the random coefficients' support, independent of draws under either estimator.) |
| seed | Random seed for draws. |
| start | Optional starting parameter vector (named or positional). |

dependence — How the two margins are linked. "famoye" (default) is Famoye/Sarmanov dependence: a single bounded association parameter (lam), admissible interval frozen at the starting values. A copula() object joins the margins with a discrete copula instead – copula("frank"), copula("normal") (Gaussian), or copula("kimeldorf") (Clayton) – each with its own native dependence parameter, always estimated (copula()'s par argument is for the simulators only). Copula evaluation costs more per iteration than Famoye at comparable draws/n, and the dependence family also changes peak memory under this engine – see max_workload at rpbnb_control() for the measured per-family weights. "independence" – two separate NB2 margins, no association parameter at all – is only available here; the classic engine (fit_rpbnb()) has no independence path for random-parameter models (use fit_bnb() for a fixed-coefficient independence model). Whichever dependence is chosen, a random coefficient on a 0/1 dummy regressor present in both equations is weakly identified (NB dispersion trades off against the random-coefficient scale); prefer a continuous regressor for a shared random coefficient when one is available.

control — An rpbnb_control() object (rpbnb_tmb_control() is a retained alias that returns the same object). One control object serves every estimator in the package; settings this engine does not read – se_method, hessian, compute_se, method, hess_eps, hess_r, draws_hessian – are ignored and listed by print()/summary() of the fit rather than rejected.

inference — Inference storage: "full" for a full covariance, "diag" for standard errors only, or "none" to skip Hessian calculations. In diagonal mode, vcov() returns NA for covariance cross-terms.

keep — Retained fit state: "postfit" keeps data needed for marginal effects, "compact" keeps estimates only, and "full" also retains the TMB objective and responses. Low-level diagnostics that access fit$obj require "full".

poisson_1, poisson_2 — Fit the corresponding margin at its exact Poisson limit (NB2 dispersion m = 0, held fixed). Available for every dependence structure, copulas included. This is the remedy for a dispersion that collapses toward zero on its own: at m ~ 1e-7 the NB2 size 1/m is ~1e7 and the marginal CDF is computed as a difference of logarithms agreeing to eleven digits, which leaves the value intact but the curvature – and so the standard errors – numerically worthless. Pinning the limit removes the parameter instead of estimating it into that regime.

method — Estimator for the random-coefficient integral. "sml" (default) uses simulated maximum likelihood over Halton draws. "laplace" uses TMB's Laplace approximation with a sparse Hessian over one latent vector per observation, which removes draws from the memory cost and makes tape size scale with nrow(data) alone.

The two are different approximations to the same integral. They agree asymptotically but need not agree closely on any given dataset, so a Laplace fit is not a drop-in reproduction of an SML fit. "laplace" supports "normal" and "lognormal" random coefficients only, and requires at least one random coefficient.

summary() reports AIC and BIC for either estimator, but under "laplace" they are computed from an approximated marginal likelihood rather than the exact one, so an AIC from a Laplace fit is not meaningful to compare against an AIC from an SML fit of the same model.

disable_parallel_gaussian — Opt-out that restricts a Gaussian-copula (dependence = copula("normal")) fit to one thread. Default FALSE: the requested control$n_cores / control$parallel_tape

are honored for every dependence family, Gaussian included. Multithreaded Gaussian evaluation used to crash the R process (SIGSEGV) whenever a fit realized more threads than an earlier fit in the same session – TMB's REGISTER_ATOMIC cache sizes its per-thread array once, at first initialization – and Gaussian fits were therefore capped to one thread by default. That defect was fixed in 0.4.4 (src/rpbnb_tmb.cpp sizes the atomic for the machine's full processor count at initialization; verified at 2/4/8/16 threads), and since 0.4.6 parallel evaluation is the default. Setting disable_parallel_gaussian = TRUE restores the old single-thread behaviour – a kill-switch for an unusual TMB/OpenMP build where the registered atomic still misbehaves. Ignored (threads never restricted) for every other dependence structure.

force_parallel_gaussian
: Deprecated (pre-0.4.6 polarity); use disable_parallel_gaussian instead. Multithreaded Gaussian evaluation is now the default, so TRUE (the old opt-in) is a no-op beyond its deprecation warning, and an explicit FALSE – which used to select the single-thread cap – is honored as disable_parallel_gaussian = TRUE. Any non-NULL value warns.

.fixed
: Internal. A named numeric vector of parameters (in the optimization parameterization, e.g. c("log_sd1:x" = -20)) to pin at the supplied values and hold fixed during estimation, so they leave the free-parameter count and logLik()'s df. Used by rpbnb_tmb_boundary_tests() to construct scale-zero restricted fits; not intended for direct use. The classic engine's fit_rpbnb() carries an identically named argument.

## Value

An object of class rpbnb_tmb_fit. The sdreport field is a compact package-owned summary and does not retain a second TMB tape.

optimizer is the nlminb() return value, with three fields added: restarts counts how many times the solve was restarted from its own answer to clear control$gradtol, max_abs_gradient is the score norm actually achieved, and gradient_tolerance is the threshold it was judged against. convergence and message come from nlminb's relative *function* test and can report success at a point that is not stationary, so max_abs_gradient is the field to check before trusting a standard error.

method records which estimator produced the fit, echoing the method argument ("sml" or "laplace"). Both print() and summary() display it, so two fits of the same model under different estimators are distinguishable in their printed output.

boundary_report is a character vector naming the reported quantities whose estimates are pinned against an implementation bound – the frozen Famoye interval, the Frank overflow guard, or an exp() clamp – or whose delta-method derivative has collapsed numerically. Those estimates are set by the implementation rather than identified by the data, so their standard errors and covariances are reported as NA and a warning is raised. An empty vector means no such constraint bound.

boundary_sides names, for each entry of boundary_report, which end was reached: "lower", "upper", or "degenerate" when the derivative collapsed rather than a bound being hit. The two sides call for opposite remedies – a lower dispersion clamp means the margin is effectively Poisson, an upper one means it is degenerately over-dispersed – so this is retained on the fit rather than only mentioned in the warning.

lambda_bounds is a named numeric vector c(lower =, upper =) giving the admissible Famoye dependence interval, and NULL for every other dependence structure. The bounds *used by the likelihood* are computed once at the starting values and held fixed for the whole fit. A lam estimate at either end is therefore an artefact of the starting values rather than a property of the data, which

is why the field is exposed. rpbnb_tmb_dependence_profile reports a likelihood-based interval where the delta-method standard error collapses to NA, but for Famoye that interval is mapped through this same frozen box and so cannot escape it: widen the box by refitting from better starting values before treating such an interval as informative.

lambda_bounds_at_optimum is the interval admissible at the *fitted* parameters, recomputed after optimization for validation only — it never enters the likelihood. lambda_admissible records whether the fitted lam (mapped through the frozen lambda_bounds, i.e. the value the objective actually used) lies inside it; FALSE means the optimizer left the region where the joint pmf is a valid probability model, a warning was raised, and the fit should not be interpreted — refit from starting values closer to the optimum. When the bound is parameter-independent (normal or lognormal coefficients loaded in both margins, where it is the constant c(-1, 1)), the two intervals coincide and the flag is trivially TRUE. Both fields are NULL/NA outside Famoye dependence.

## Examples

```
## Not run:
sim <- simulate_rpbnb_tmb(n = 300,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb_tmb(y1 ~ x1, y2 ~ x1, data = sim$data, draws = 100)
coef(fit)

## End(Not run)
```

---

# lr_test *Likelihood-ratio test between two nested model fits*

---

## Description

Compares a restricted fit against a full (nesting) fit by the likelihood-ratio statistic. Works for any fit_bnb() / fit_rpbnb() objects, since both carry a logLik() with a "df" attribute equal to the number of estimated parameters.

## Usage

```
lr_test(restricted, full, boundary = FALSE)
```

## Arguments

| | |
|---|---|
| restricted | The smaller (restricted) fit – fewer estimated parameters. |
| full | The larger (full) fit that nests restricted. |
| boundary | Logical. When the restriction pins a variance/dispersion-type parameter (a random-coefficient SD, or NB2 dispersion m) to its zero boundary, the null distribution is not a plain chi-square. Set boundary = TRUE to use the 50:50 mixture of chisq(df) and chisq(df - 1) (Self & Liang, 1987); for a single boundary parameter (df = 1) this halves the naive p-value. Default FALSE (ordinary interior restriction, e.g. dropping a fixed covariate). The mixture is exact only for a single parameter on the boundary; simultaneous boundary restrictions of several parameters need different weights and are not handled. |

## Details

The restricted model is one you fit yourself with a term removed – for example dropping a name from `random_1` (testing a random-coefficient SD), or a plain NB / independence fit (testing an NB2 dispersion). This is the statistically appropriate replacement for the Wald z/p that the natural-scale summary suppresses on positive scale/dispersion parameters.

A likelihood-ratio test requires two *maximized* likelihoods. When either argument is a package fit (`bnb_fit` / `rpbnb_fit`) that records a failed optimization (`convergence$converged = FALSE`), `lr_test()` errors rather than returning a p-value from an unfinished fit – the sign of the statistic cannot establish convergence. Generic objects that only carry a `logLik()` (no convergence record) are still accepted, but their convergence cannot be validated and is the caller's responsibility.

## Value

An object of class `rpbnb_lrtest` with the LR `statistic`, degrees of freedom `df`, `p.value`, the two log-likelihoods and their df, and the `boundary` flag. Has a `print` method.

## Famoye caveat for bounded random-coefficient distributions

Each Famoye fit maximizes its likelihood over an admissible lambda interval frozen at that fit's own starting values. With normal or lognormal random coefficients loaded in both margins the interval is the constant `c(-1, 1)` for every fit, so the comparison is unaffected. With uniform or triangular coefficients (or a single varying margin) the bound moves with the parameters, so two fits frozen at different starting values maximize over slightly different lambda ranges and the LR statistic inherits that second-order discrepancy. Check `lambda_admissible` on both fits before relying on a comparison in that regime.

## References

Self, S. G. and Liang, K.-Y. (1987). Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions. *JASA* 82(398), 605–610.

## Examples

```
sim <- simulate_rpbnb(n = 600,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
ctrl <- rpbnb_control(compute_se = FALSE)
full <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                  draws = 100, control = ctrl)
rest <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data,
                  draws = 100, control = ctrl)   # no random coefficient
lr_test(rest, full, boundary = TRUE)             # test sd(x1) = 0
```

---

`plot.bnb_fit` *Residual diagnostic plots for a bivariate NB model*

---

## Description

Four base-graphics panels per margin: residuals-vs-fitted, a normal QQ plot of the randomized quantile residuals, a histogram of the RQR with an N(0,1) overlay, and a scale-location plot. The QQ and histogram panels always use RQR (only these are approximately N(0,1) under a correct count model).

## Usage

```
## S3 method for class 'bnb_fit'
plot(
  x,
  margin = c("both", "y1", "y2"),
  which = 1:4,
  resid_type = "quantile",
  seed = NULL,
  ...
)
```

## Arguments

| | |
|---|---|
| x | A bnb_fit object from fit_bnb(). |
| margin | Which margin to plot: "both" (default), "y1", or "y2". |
| which | Integer subset of panels 1:4 (1 = residuals-vs-fitted, 2 = QQ, 3 = histogram, 4 = scale-location). |
| resid_type | Residual type for panels 1 and 4: "quantile" (default), "pearson", "deviance", or "response". |
| seed | Optional integer seed for the RQR randomization. |
| ... | Unused. |

## Value

NULL, invisibly (called for the side effect of drawing).

---

# plot.rpbnb_fit *Residual diagnostic plots for a random-parameter bivariate NB model*

---

## Description

Four base-graphics panels per margin, as for plot.bnb_fit(), built on the mixture-based randomized quantile residuals. resid_type = "deviance" is not available for rpbnb_fit.

## Usage

```
## S3 method for class 'rpbnb_fit'
plot(
  x,
  margin = c("both", "y1", "y2"),
  which = 1:4,
  resid_type = "quantile",
  seed = NULL,
  ...
)
```

### Arguments

| | |
|---|---|
| x | An rpbnb_fit object from fit_rpbnb(). |
| margin | Which margin to plot: "both" (default), "y1", or "y2". |
| which | Integer subset of panels 1:4. |
| resid_type | Residual type for panels 1 and 4: "quantile" (default), "pearson", or "response". |
| seed | Optional integer seed for the RQR randomization. |
| ... | Unused. |

### Value

NULL, invisibly.

---

## predict.rpbnb_tmb_fit *Predict from a fitted bivariate count model*

---

### Description

Predictions integrate over the retained simulation draws when random coefficients are present. Link predictions are the log of the integrated response mean, so exp(predict(fit, type = "link")) equals response predictions.

### Usage

```
## S3 method for class 'rpbnb_tmb_fit'
predict(
  object,
  newdata = NULL,
  type = c("response", "link"),
  which = c("both", "y1", "y2"),
  ...
)
```

### Arguments

| | |
|---|---|
| object | A fitted rpbnb_tmb_fit object. |
| newdata | Optional data frame. If omitted, the retained fitting design is used; compact fits require newdata. |
| type | Either "response" or "link". |
| which | Return both margins or only "y1" or "y2". |
| ... | Reserved for future use. |

### Value

A two-column numeric matrix for which = "both", otherwise a numeric vector.

## residuals.bnb_fit *Residuals for a bivariate NB model*

### Description

Per-margin residuals for a fixed-coefficient `fit_bnb()` model. Each margin is NB2 with fitted mean `mu` and dispersion `m` (`size = 1/m`). `"quantile"` returns randomized quantile residuals (Dunn & Smyth 1996), which are approximately N(0,1) under a correct model and are the recommended residual for normality-style diagnostics on count data.

### Usage

```
## S3 method for class 'bnb_fit'
residuals(
  object,
  type = c("quantile", "pearson", "deviance", "response"),
  margin = c("both", "y1", "y2"),
  seed = NULL,
  ...
)
```

### Arguments

| | |
|---|---|
| `object` | A `bnb_fit` object from `fit_bnb()`. |
| `type` | Residual type: `"quantile"` (default), `"pearson"`, `"deviance"`, or `"response"`. |
| `margin` | Which margin: `"both"` (default), `"y1"`, or `"y2"`. |
| `seed` | Optional integer seed for the quantile-residual randomization (ignored for other types); does not disturb the caller's RNG stream. |
| `...` | Unused. |

### Value

A numeric vector for a single margin, or a two-column data frame (`y1`, `y2`) for `margin = "both"`.

## residuals.rpbnb_fit *Residuals for a random-parameter bivariate NB model*

### Description

Per-margin residuals for an `fit_rpbnb()` model. Each margin is a mixture of NB2 distributions over the random-coefficient draws. `"quantile"` returns randomized quantile residuals (Dunn & Smyth 1996) from the exact mixture predictive CDF (the recommended residual); `"pearson"` uses the exact mixture marginal variance. `"deviance"` is not defined for the mixture and errors.

## Usage

```
## S3 method for class 'rpbnb_fit'
residuals(
  object,
  type = c("quantile", "pearson", "deviance", "response"),
  margin = c("both", "y1", "y2"),
  seed = NULL,
  ...
)
```

## Arguments

| | |
|---|---|
| object | An rpbnb_fit object from fit_rpbnb(). |
| type | Residual type: "quantile" (default), "pearson", or "response". "deviance" is not supported for rpbnb_fit. |
| margin | Which margin: "both" (default), "y1", or "y2". |
| seed | Optional integer seed for the quantile-residual randomization; does not disturb the caller's RNG stream. |
| ... | Unused. |

## Value

A numeric vector for a single margin, or a two-column data frame (y1, y2) for margin = "both".

---

rpbnb *Fit a random-parameter bivariate NB model with either engine*

---

## Description

A common front end over the package's two estimation engines. engine = "classic" calls fit_rpbnb() (Rcpp/OpenMP simulated likelihood, maxLik BFGS); engine = "tmb" calls fit_rpbnb_tmb() (TMB automatic differentiation, nlminb with restart polish). Both fitters remain exported and can be called directly; with standardize = FALSE (the default) this wrapper adds nothing to the fit itself and returns exactly what the chosen fitter returns.

## Usage

```
rpbnb(
  formula_1,
  formula_2,
  data,
  engine = c("classic", "tmb"),
  random_1 = NULL,
  random_2 = NULL,
  draws = 400,
  seed = 1234,
  start = NULL,
  dependence = "famoye",
  poisson_1 = FALSE,
  poisson_2 = FALSE,
```

```
  standardize = FALSE,
  continuous_vars = NULL,
  boundary_tests = FALSE,
  boundary_draws = NULL,
  control = NULL,
  ...
)
```

## Arguments

| | |
|---|---|
| formula_1, formula_2 | Model formulas for the two count responses. |
| data | A data frame containing the model variables. |
| engine | Which fitter estimates the model. "classic" (default) calls fit_rpbnb(): a multithreaded (OpenMP) Rcpp simulated-likelihood engine optimized by maxLik::maxLik(method = "BFGS") with a numerical gradient. "tmb" calls fit_rpbnb_tmb(): it builds an automatic-differentiation tape with TMB (Template Model Builder) and optimizes it with stats::nlminb plus a restart polish – the exact gradient this gives tends to converge faster and more reliably than the classic engine's numerical one, especially under copula dependence. "tmb" is also the only engine that accepts dependence = "independence", the Laplace estimator (method = "laplace", passed via ... – see the "estimator" discussion below and fit_rpbnb_tmb()'s method argument), and memory-aware draw chunking for large draws; "classic" is the only one that accepts an offset() term in a formula. Both engines share one rpbnb_control() object and this function's cross-engine argument checking, but otherwise a fit is exactly what a direct call to the chosen fitter would return – see "Which arguments go with which engine" below for the full compatibility table, and the Return section for how the fit class depends on engine. |
| random_1, random_2 | Random-coefficient specifications for each equation. |
| draws | Number of simulation draws. |
| seed | Random seed for the draw sequence. |
| start | Optional named or unnamed starting values. |
| dependence | How the two margins are linked. "famoye" (default) is Famoye/Sarmanov dependence: a single bounded association parameter (lam), with an admissible interval frozen at the starting values (see "Which arguments go with which engine" below). A copula() object joins the margins with a discrete copula instead – copula("frank"), copula("normal") (Gaussian), or copula("kimeldorf") (Clayton) – each with its own native dependence parameter, always estimated (copula()'s par argument is for the simulators only, not the fitters). Copula evaluation costs more per iteration than Famoye at comparable draws/n (discrete-copula pmf plus a per-draw NB CDF corner), and under engine = "tmb" the dependence family also changes peak memory – see max_workload at rpbnb_control() for the measured per-family weights. "independence" – two separate NB2 margins, no association parameter at all – is engine = "tmb" only; the fixed-coefficient fit_bnb() supports it under engine = "classic", but the random-parameter classic engine (fit_rpbnb()) does not. Whichever dependence is chosen, a random coefficient on a 0/1 dummy regressor present in both equations is weakly identified (NB dispersion trades off against the random-coefficient scale); prefer a continuous regressor for a shared random coefficient when one is available. |

poisson_1, poisson_2
: Restrict the corresponding margin to its Poisson limit.

standardize
: Centre and scale continuous predictors before fitting, and display fitted coefficients back-transformed to their original units (see "Automatic centring and scaling" above). Default FALSE.

continuous_vars
: Optional character vector overriding automatic continuous-predictor detection when standardize = TRUE. Must be columns of data; ignored when standardize = FALSE.

boundary_tests
: Which groups of parameters to LR-test after fitting, via rpbnb_boundary_tests() (engine = "classic") or rpbnb_tmb_boundary_tests() (engine = "tmb"); the result is attached as $boundary_tests and summary()/print() show the test (rather than NA, or rather than a Wald z) on the corresponding rows. Accepts:

  - FALSE (default) or "none" – run nothing. Each restricted refit costs roughly another full fit.
  - TRUE – c("sd", "dispersion"), the two boundary-null groups. This is what TRUE has always meant and it does not silently grow.
  - a character vector, any of "sd" (random-coefficient scales), "dispersion" (the NB2 overdispersions m1, m2), "dependence" (the association parameter), or "all" for all three.

  So boundary_tests = "dispersion" tests overdispersion only, boundary_tests = c("dispersion", "dependence") tests overdispersion and association without paying for one refit per random-coefficient scale, and boundary_tests = "all" tests everything. See "Boundary LR tests" below.

boundary_draws
: Number of Halton simulation draws for the boundary tests' restricted refits, engine = "tmb" only. NULL (default) uses the main fit's draws. Ignored when no boundary test was requested (nothing to apply it to); an error if supplied under engine = "classic" alongside a boundary test (see "Boundary LR tests" above – that engine has no draws to override).

control
: An rpbnb_control() object – one object for both engines, as of 0.4.1 (rpbnb_tmb_control() is a retained alias that returns the same thing). Settings the chosen engine does not read are ignored and listed by print()/summary() of the fit; iterlim and print_level, whose defaults differ between the engines, resolve to the chosen engine's own default unless you set them. Fields sharing a name are still not translated: iterlim is a maxLik BFGS limit under engine = "classic" and an nlminb limit under "tmb", and n_cores is worker processes versus OpenMP threads.

...
: Further arguments passed to the selected fitter. Names are validated against that fitter's formals; an argument belonging to the other engine, or an unrecognised name, is an error. Exception: the TMB tuning knobs method, disable_parallel_gaussian, and the deprecated force_parallel_gaussian are dropped with a warning (not an error) under engine = "classic", so a call can switch engines without stripping them.

  The estimator, method (engine = "tmb" only, default "sml"), picks how the random-coefficient integral is approximated. "sml" is simulated maximum likelihood over draws Halton points – the same integral the classic engine approximates, but with an exact automatic-differentiation gradient. "laplace" instead uses TMB's Laplace approximation: a sparse-Hessian integration over one latent vector per observation that removes draws from the memory cost entirely (tape size then scales with nrow(data) alone, not nrow(data) * draws),

at the cost of requiring at least one random coefficient and restricting it to "normal"/"lognormal" ("uniform" and "triangular" error under Laplace). The two estimators agree asymptotically but are different approximations to the same integral – not interchangeable point-for-point on a given dataset, and their summary() AIC/BIC are not comparable to each other for the same reason. See fit_rpbnb_tmb()'s method argument for the full detail, including how boundary tests and predict() behave differently under each.

## Details

What it does add is argument checking across the two APIs. The engines do not take the same arguments, and passing one engine's argument to the other is a mistake that is easy to make and expensive to notice — a standardized coefficient table printed under an "original units" heading looks perfectly plausible. Every extra argument is therefore matched by name against the selected fitter's own formals, and anything that does not belong is an error rather than a silently ignored ... entry.

## Value

The engine-native fit object, identical to what a direct call to the underlying fitter would return: an object of class rpbnb_fit for engine = "classic", or rpbnb_tmb_fit for engine = "tmb". The class therefore depends on engine; test with inherits(fit, "rpbnb_tmb_fit") if you need to branch. No wrapper class is introduced, so every existing S3 method and post-estimation function works unchanged. With standardize = TRUE, two extra fields are attached – $scaling and $continuous_vars – and print()/summary() use them to display original-units coefficients. With any boundary_tests group requested, a third field $boundary_tests (the rpbnb_boundary_tests() result) is attached, and print()/summary() use it to show the LR test on the corresponding rows. Both fitters also attach $control_ignored / $control_engine, the control settings the chosen engine did not read. Nothing else on the object changes.

## Automatic centring and scaling

standardize = TRUE automates the pattern in inst/dev/rpbnb_frank_open.R and inst/dev/tmb_rpbnb_frank_oper continuous predictors are centred and scaled (mean 0, SD 1) before fitting, which keeps a bounded random-coefficient carrier from acting as a disguised random intercept (see those scripts' headers) and fixes the design matrix's conditioning when regressors span very different ranges. Continuous predictors are identified automatically — numeric, non-factor columns used by either formula with more than two distinct values, so 0/1 (or any two-level numeric) indicators are left alone — or supplied explicitly via continuous_vars. Variables that appear only inside an offset() are never standardized.

The fitted design itself (the stored X1/X2, mu1/mu2, simulation draws) stays on the standardized scale, exactly as in the two scripts above, so predict(), marginal effects, and boundary/LR tests keep working unchanged. Only the coefficient table print() and summary() display is affected: it is back-transformed to the covariates' original units by the exact affine chain rule (no refit, no numerical differentiation) and is the *only* coefficient table shown — there is no separate standardized-scale table to reconcile. coef()/vcov() still return the standardized-scale values that match the stored design; call .rpbnb_orig_units() (internal, mirrors what print() displays) if the numeric original-units vector is needed directly. The scaling actually used is stored on the fit as $scaling (a named list of c(center=, scale=)) and $continuous_vars.

## Boundary LR tests

boundary_tests runs a likelihood-ratio test on the fit as soon as it converges (against the same

data the fit itself used – the standardized copy, when `standardize = TRUE`) and attaches the result as `$boundary_tests`. It is a switch over three independent groups, so the cost is paid only for the parameters actually in question:

| `boundary_tests` | tests | restricted refits |
|---|---|---|
| `FALSE` / `"none"` | nothing | 0 |
| `TRUE` | `c("sd", "dispersion")` | one per scale + one per free `m` |
| `"sd"` | random-coefficient scales | one per scale |
| `"dispersion"` | overdispersions `m1`, `m2` | one per free `m` |
| `"dependence"` | the association parameter | 1 |
| `"all"` | all three groups | all of the above |

The random-coefficient SDs and the NB2 dispersions (`m1`, `m2`) have a null that sits on the boundary of the parameter space, so an ordinary Wald `z`/`p` does not test it; the boundary test refits each restricted model instead and `summary()`/`print()` show that test's `LR`/`df`/`p` for those rows in the natural-scale block, in place of the `NA` they would otherwise carry.

The **dependence** test is the odd one out and is therefore opt-in even under `boundary_tests = TRUE`. Its null is "no association", i.e. the independence model, which for Famoye (`lam`), Frank (`theta`), and the Gaussian copula (`rho`) sits in the *interior* of the parameter space – the Wald `z` those rows already show is valid, and the LR statistic is an ordinary chi-square(1), not the 50:50 boundary mixture. Only Clayton / Kimeldorf (`theta > 0`) has a genuine boundary null there. Requesting it replaces the dependence row's Wald `z`/`p` with the LR test on both engines, since the two answer the same question.

Both engines test the same parameters via `rpbnb_boundary_tests()` (`engine = "classic"`) or `rpbnb_tmb_boundary_tests()` (`engine = "tmb"`). They differ only in how each restricted fit is constructed while preserving common random numbers: for a scale, the classic engine zeroes that coefficient's draw column while the TMB engine pins its `log_sd` and maps it out of the free parameters; for the dependence parameter, the TMB engine refits with its own `dependence = "independence"` family while the classic engine (which has no such fitter) pins the working-scale dependence parameter at its family's independence value.

A `message()` reports how many restricted refits are about to run before they start, unless `control$print_level` is `0`; suppress it with `suppressMessages()` if needed.

Under `engine = "tmb"` with `method = "laplace"`, a restricted refit that fails to converge is automatically retried with both sides of that one LR estimated by `method = "sml"` rather than reported `NA` – some restrictions leave Laplace no valid optimum at all (see `rpbnb_tmb_boundary_tests()`'s `sml_fallback` argument, which is where to turn this off).

`disable_parallel_gaussian` (`engine = "tmb"` only, passed via `...`) is forwarded to every restricted refit, so a Gaussian-copula fit that opted out of multithreading gets single-threaded boundary refits too – see `rpbnb_tmb_boundary_tests()`'s own `disable_parallel_gaussian` argument for why this needs forwarding at all (the fit object does not record whether the opt-out was used). The deprecated pre-0.4.6 `force_parallel_gaussian` is likewise accepted and mapped (see `fit_rpbnb_tmb()`).

Each restricted refit costs roughly as much as the original fit (more for a `copula()` dependence than for `"famoye"`; see `rpbnb_boundary_tests()`'s timing note), so this defaults to `FALSE`. `boundary_draws` (`engine = "tmb"` only) sets a `draws` for those refits other than the main fit's – e.g. more draws for a more precise boundary test without re-fitting the whole model at that `draws`. It has no classic-engine counterpart: `rpbnb_boundary_tests()` reuses the full fit's exact stored draw matrix (zeroing a column) rather than regenerating draws from a count, so passing `boundary_draws` under `engine = "classic"` is an error. For finer control still – testing only `"sd"` or only `"dispersion"`

(both engines take a `which` argument), or reusing one boundary-test run across several summaries – call `rpbnb_boundary_tests()`/`rpbnb_tmb_boundary_tests()` directly on the fit and assign its result to `fit$boundary_tests` (with `standardize = TRUE`, reconstruct the fitting-scale data first: `rpbnb:::.apply_scaling(data, fit$scaling)`).

### Which arguments go with which engine

| Argument | `engine = "classic"` | `engine =` |
|---|---|---|
| `draw_type`, `.fixed`, `.opt_draws` | yes | error |
| `inference`, `keep` | error | yes |
| `method`, `disable_parallel_gaussian`, `force_parallel_gaussian` | ignored with a warning | yes |
| `offset()` in a formula | yes | error |
| `dependence = "independence"` | error | yes |
| `boundary_draws` (non-NULL) | error | yes |
| `control` class | `rpbnb_control` | `rpbnb_c` |
| optimizer | `maxLik::maxLik(method = "BFGS")` | `stats::` |

Both engines freeze the Famoye admissible lambda interval at the starting values (so the analytic gradient is exactly the derivative of the optimized objective) and validate the fitted lambda against the interval admissible at the optimum afterwards — check `lambda_admissible` on the fit. The fixed-parameter `fit_bnb()` takes the opposite trade-off (moving bounds, admissible by construction); see the decision note in `R/bnb_likelihood.R`.

### See Also

`fit_rpbnb()`, `fit_rpbnb_tmb()`, `rpbnb_control()`, `rpbnb_tmb_control()`, `copula()`, `fit_bnb()`

### Examples

```
d <- read.csv(system.file("extdata", "rwm1984_bnb.csv", package = "rpbnb"))
fit <- rpbnb(docvis ~ outwork, hospvis ~ outwork, data = d,
             engine = "tmb", random_1 = "outwork", draws = 50)


## Not run:
# Fuller worked examples, shipped as standalone scripts under inst/ --
# run with Rscript inst/<file>.R from a source checkout, or
# Rscript system.file("<file>.R", package = "rpbnb") after install.
# Shown here for reference only (see the comment above .rpbnb_inst_examples_doc()
# in R/rpbnb.R for why none of these run under R CMD check).

# ---- inst/example_rpbnb_tmb_famoye.R: TMB, method = "sml", Famoye/Sarmanov dependence (rwm1984.csv) ----
#!/usr/bin/env Rscript
# ============================================================================
# example_rpbnb_tmb_famoye.R -- random-parameter bivariate NB (RP-BNB), TMB
# engine, method = "sml", Famoye/Sarmanov dependence
#
# Estimates the same random-parameter bivariate NB2 model as
# inst/example_rpbnb_classic.R's Famoye fit -- same data (rwm1984.csv), same
# dummy variables (see inst/example_bnb.R), same formulas, same random
# coefficient (kids, in both equations) -- but via the TMB engine
# (fit_rpbnb_tmb() / rpbnb(engine = "tmb")) instead of the classic engine, and
# with method = "sml" explicit rather than relying on it being the default.
```

```
#
# This is the Famoye/Sarmanov side of the TMB engine's SML estimator; see
# inst/example_rpbnb_tmb_sml.R for the copula-dependence counterpart (the two
# were one script until split so each dependence structure -- and its own
# boundary-test cost -- can be run independently rather than paying for both
# every time).
#
# The TMB engine has two estimators for the random-coefficient integral:
# "sml" (simulated maximum likelihood over Halton draws, the same integral
# the classic engine approximates, but with an exact automatic-differentiation
# gradient instead of a numerical one) and "laplace" (a sparse-Hessian
# approximation that removes `draws` from the memory cost -- see
# ?fit_rpbnb_tmb's `method` argument). This script is the "sml" side of that
# choice: it should give estimates close to inst/example_rpbnb_classic.R's
# (same integral, same data, same draws), not close to a Laplace fit of the
# same model, which approximates a different way and need not agree closely
# on any given dataset (see ?fit_rpbnb_tmb). The two scripts' boundary tests
# do NOT match, though: this one narrows LR_TEST to "sd" (2 refits), while
# the classic script uses boundary_tests = TRUE (sd + dispersion, 4 refits)
# -- see LR_TEST below to widen this one to match.
#
# `kids` is loaded as random in BOTH equations: under Famoye/Sarmanov
# dependence, a normal/lognormal random coefficient present in both margins
# makes the admissible lambda interval the constant c(-1, 1) (see
# ?fit_rpbnb_tmb's Return section, `lambda_bounds`), which the optimizer can
# never escape. A single-margin random coefficient does not have that
# guarantee.
#
# Run with:   Rscript inst/example_rpbnb_tmb_famoye.R
# or, after install:
#   Rscript -e 'source(system.file("example_rpbnb_tmb_famoye.R", package = "rpbnb"))'
# ============================================================================

#library(rpbnb)
devtools::load_all()

# ---- Load data -------------------------------------------------------------
d <- read.csv(system.file("extdata", "rwm1984.csv", package = "rpbnb", mustWork = TRUE))

cat("=== rwm1984.csv ===\n")
cat("Observations:", nrow(d), "\n")
cat("docvis  (doctor visits) : mean =", round(mean(d$docvis), 2),
    " var =", round(var(d$docvis), 2), "\n")
cat("hospvis (hospital visits): mean =", round(mean(d$hospvis), 2),
    " var =", round(var(d$hospvis), 2), "\n\n")

# ---- Dummy variables from the raw categorical columns ----------------------
# Same derivation as inst/example_bnb.R -- see that script for the full
# explanation. postHS: schooling beyond the lowest track (edlevel >= 2). MarM/
# MarF/SinM/SinF: sex x marital-status, four mutually exclusive dummies; SinM
# (single male) is left out of the formulas below as the reference category.
d$postHS <- as.integer(d$edlevel >= 2)
d$MarM <- as.integer(d$married == 1 & d$female == 0)  # married male
d$MarF <- as.integer(d$married == 1 & d$female == 1)  # married female
d$SinM <- as.integer(d$married == 0 & d$female == 0)  # single male (reference)
d$SinF <- as.integer(d$married == 0 & d$female == 1)  # single female
```

```
f1 <- docvis  ~ outwork + kids + postHS + MarM + MarF + SinF
f2 <- hospvis ~ outwork + kids + postHS + MarM + MarF + SinF

DRAWS <- 1000
SEED  <- 1234
# rpbnb_control()'s n_cores is the OpenMP thread count for both TMB estimators
# (see ?rpbnb_control); leave 1 core free for the rest of the system. TMB's
# se_method-equivalent is the `inference` argument to fit_rpbnb_tmb() (full
# covariance by default), not a control field -- control$se_method is a
# classic-engine-only knob and is ignored here.
N_CORES <- max(1L, parallel::detectCores() - 1L)
# Which boundary_tests group(s) to run -- see ?rpbnb's boundary_tests table.
# "sd" tests only the random-coefficient scales (sd1:kids, sd2:kids); add
# "dispersion" (or pass TRUE for both) to also test m1/m2, at the cost of two
# more restricted refits.
LR_TEST <- "sd"
ctrl <- rpbnb_control(n_cores = N_CORES)
cat("Using", N_CORES, "OpenMP threads,", DRAWS, "draws.\n\n")

# ---- Famoye/Sarmanov RP-BNB, TMB engine, SML ----------------------------------
# boundary_tests = LR_TEST runs rpbnb_tmb_boundary_tests() on the converged fit
# and attaches it as fit$boundary_tests, so summary() below fills in the
# sd1:kids/sd2:kids LR/df/p columns instead of leaving them blank -- no
# separate rpbnb_tmb_boundary_tests() call needed.
cat("=== Fitting RP-BNB (TMB engine, method = sml, Famoye/Sarmanov, random kids) ===\n")
t_famoye <- system.time(
  fit_famoye <- rpbnb(f1, f2, data = d, engine = "tmb", method = "sml",
                      random_1 = "kids", random_2 = "kids",
                      draws = DRAWS,
                      seed = SEED,
                      dependence = "famoye",
                      control = ctrl,
                      boundary_tests = LR_TEST)
)[["elapsed"]]
cat(sprintf("Estimation finished in %.1f s\n\n", t_famoye))
print(summary(fit_famoye))

# The same table summary() just read from, printed standalone: H0: sd(kids) = 0
# in each equation (kids' coefficient is not actually random). This null sits
# on the boundary of the parameter space, which is why summary() reports no
# Wald z/p for it without this test.
cat("\n=== Boundary LR tests (sd1:kids, sd2:kids) ===\n")
print(fit_famoye$boundary_tests)

# ---- Predicted means for a couple of profiles ---------------------------------
# Two married women, one working, one not, otherwise identical (one kid,
# post-Hauptschule schooling); MarM/SinF both 0 -> MarF = 1. Unlike the
# classic engine's predict.rpbnb_fit() (columns mu1/mu2), predict.rpbnb_tmb_fit()
# returns a matrix with columns y1/y2.
newdata <- data.frame(outwork = c(0, 1), kids = c(1, 1), postHS = c(1, 1),
                      MarM = c(0, 0), MarF = c(1, 1), SinF = c(0, 0))
cat("\n=== Predicted means (Famoye/Sarmanov fit) ===\n")
print(cbind(newdata, predict(fit_famoye, newdata = newdata)))

# ---- inst/example_rpbnb_tmb_sml.R: TMB, method = "sml", copula dependence (rwm1984.csv) ----
#!/usr/bin/env Rscript
# ============================================================================
```

```
# example_rpbnb_tmb_sml.R -- random-parameter bivariate NB (RP-BNB), TMB
# engine, method = "sml", copula dependence
#
# Estimates a random-parameter bivariate NB2 model for the two count outcomes
# in the German health-care utilization panel (rwm1984.csv) under copula
# dependence (COPULA_FAMILY below), via the TMB engine's SML estimator. Same
# data, dummy variables (see inst/example_bnb.R), formulas, and random
# coefficient (kids, in both equations) as inst/example_rpbnb_tmb_famoye.R --
# that script covers Famoye/Sarmanov dependence; this one was split out of it
# so each dependence structure -- and its own boundary-test cost -- can be run
# independently rather than paying for both every time.
#
# The TMB engine has two estimators for the random-coefficient integral:
# "sml" (simulated maximum likelihood over Halton draws, the same integral
# the classic engine approximates, but with an exact automatic-differentiation
# gradient instead of a numerical one) and "laplace" (a sparse-Hessian
# approximation that removes `draws` from the memory cost -- see
# ?fit_rpbnb_tmb's `method` argument). This script is the "sml" side of that
# choice, not close to a Laplace fit of the same model, which approximates a
# different way and need not agree closely on any given dataset (see
# ?fit_rpbnb_tmb). At COPULA_FAMILY = "normal" it should also give estimates
# close to inst/example_rpbnb_classic.R's copula fit (same integral, same
# data, same draws) -- that script only demonstrates the Gaussian copula, so
# the comparison holds only for that one family, not for whichever
# COPULA_FAMILY happens to be set below. The two scripts' boundary tests do
# NOT match, either: this one narrows LR_TEST to "sd" (2 refits), while the
# classic script uses boundary_tests = TRUE (sd + dispersion, 4 refits) -- see
# LR_TEST below to widen this one to match.
#
# `kids` is loaded as random in BOTH equations, for consistency with
# inst/example_rpbnb_tmb_famoye.R and to avoid identifying the random
# coefficient off only one margin's variation. Note this cuts against
# ?fit_rpbnb_tmb's own advice for the copula path specifically: `kids` is a
# 0/1 dummy in this data, and a random coefficient on a dummy regressor is
# weakly identified under copula dependence (NB dispersion trades off against
# the random-coefficient scale) -- kept anyway here so the two scripts'
# specifications line up; a continuous regressor would identify more cleanly.
#
# If the full model does not converge -- "false convergence (8)" with a
# non-positive-definite Hessian and NA scale/dispersion SEs -- that is the
# weak-identification signature of a 0/1 dummy (kids) as the random
# coefficient in both equations under a Gaussian copula, NOT an
# optimizer-settings problem. A convergence gate after the fit (the
# "Convergence gate" block) stops with an actionable diagnostic: make the
# random coefficient a CONTINUOUS regressor in at least one equation, or load
# kids in only one equation, or use a dependence structure that identifies the
# scale better. No restarts/iterlim/rel.tol change manufactures the missing
# curvature.
#
# Run with:   Rscript inst/example_rpbnb_tmb_sml.R
# or, after install:
#   Rscript -e 'source(system.file("example_rpbnb_tmb_sml.R", package = "rpbnb"))'
# ============================================================================

library(rpbnb)
#devtools::load_all()
```

```
# ---- Load data -------------------------------------------------------------
d <- read.csv(system.file("extdata", "rwm1984.csv", package = "rpbnb", mustWork = TRUE))

cat("=== rwm1984.csv ===\n")
cat("Observations:", nrow(d), "\n")
cat("docvis  (doctor visits) : mean =", round(mean(d$docvis), 2),
    " var =", round(var(d$docvis), 2), "\n")
cat("hospvis (hospital visits): mean =", round(mean(d$hospvis), 2),
    " var =", round(var(d$hospvis), 2), "\n\n")

# ---- Dummy variables from the raw categorical columns ----------------------
# Same derivation as inst/example_bnb.R -- see that script for the full
# explanation. postHS: schooling beyond the lowest track (edlevel >= 2). MarM/
# MarF/SinM/SinF: sex x marital-status, four mutually exclusive dummies; SinM
# (single male) is left out of the formulas below as the reference category.
d$postHS <- as.integer(d$edlevel >= 2)
d$MarM <- as.integer(d$married == 1 & d$female == 0)  # married male
d$MarF <- as.integer(d$married == 1 & d$female == 1)  # married female
d$SinM <- as.integer(d$married == 0 & d$female == 0)  # single male (reference)
d$SinF <- as.integer(d$married == 0 & d$female == 1)  # single female

f1 <- docvis  ~ outwork + kids + postHS + MarM + MarF + SinF
f2 <- hospvis ~ outwork + kids + postHS + MarM + MarF + SinF

DRAWS <- 500
SEED  <- 1234
# Which copula to fit. copula() accepts exactly these three families;
# everything downstream (labels, the reported native parameter, Kendall's
# tau) adapts to whichever is set here.
#   "frank"     -- theta on the whole real line, no tail dependence
#   "normal"    -- Gaussian, rho in (-1, 1)
#   "kimeldorf" -- Kimeldorf-Sampson (Clayton), theta > 0, lower-tail dependence
COPULA_FAMILY <- "normal"

# rpbnb_control()'s n_cores is the OpenMP thread count for both TMB estimators
# (see ?rpbnb_control); leave 1 core free for the rest of the system. TMB's
# se_method-equivalent is the `inference` argument to fit_rpbnb_tmb() (full
# covariance by default), not a control field -- control$se_method is a
# classic-engine-only knob and is ignored here.
N_CORES <- max(1L, parallel::detectCores() - 1L)

# Which boundary_tests group(s) to run -- see ?rpbnb's boundary_tests table.
# "sd" tests only the random-coefficient scales (sd1:kids, sd2:kids); add
# "dispersion" (or pass TRUE for both) to also test m1/m2, at the cost of two
# more restricted refits.
LR_TEST <- "sd"

# ---- TMB engine control ----------------------------------------------------
# One rpbnb_control() object drives every estimator; here only the TMB fields
# below matter (n_cores, max_threads, max_workload, tape_chunks), the rest are
# resolved or ignored for this fit. See ?rpbnb_control for the full field
# reference and ?fit_rpbnb_tmb for the memory/chunking model.
ctrl <- rpbnb_control(
  n_cores = N_CORES,
  # OpenMP thread count for both TMB estimators (see ?rpbnb_control). Left 1
  # core free above via N_CORES.
  max_threads = N_CORES,
```

```
  # Ceiling on OpenMP threads permitted for one TMB fit. NULL would default to
  # n_cores; set equal here so the ceiling never binds below the request.
  max_workload = rpbnb_tmb_max_workload(budget_gib = 32),
  # Memory guard: the largest weighted observation-draw count the TMB tape may
  # build before the fit is refused or chunked. Weighted workload is
  # n * draws * family_weight (frank = 3.6, the heaviest family) with peak at
  # ~12083 bytes/unit, so this budgets a ~32 GiB peak for one fit. Inf disables
  # the guard entirely.
  tape_chunks = NULL,
  # SML-only: split the draws into this many chunks, replayed over one smaller
  # TMB tape, cutting peak tape memory to nrow * ceiling(DRAWS / chunks) at the
  # cost of somewhat slower gradient evaluations. A fixed integer must be 1L or
  # greater and not exceed draws (validated at fit time in .resolve_tape_chunks()).
  #
  # NOTE: NULL here (the default) auto-selects the smallest sufficient chunk
  # count when the weighted workload exceeds max_workload, or 1L (no chunking)
  # when it does not -- so with max_workload pinned to the 32 GiB budget above,
  # NULL picks the layout on its own from that budget. Pass a fixed integer
  # (e.g. 4L) to pin the layout regardless of the auto-threshold.
  #
  # Any chunked fit (value > 1L, including an auto-picked one) has no taped
  # Hessian, so profile-based inference (confint(method = "profile"),
  # rpbnb_tmb_dependence_profile()) falls back to Wald with a warning; default
  # Wald/optimHess is unaffected.
)
cat("Using", N_CORES, "OpenMP threads,", DRAWS, "draws.\n\n")

# ---- Copula RP-BNB, TMB engine, SML ------------------------------------------
# boundary_tests = LR_TEST runs rpbnb_tmb_boundary_tests() on the converged fit
# and attaches it as fit$boundary_tests, so summary() below fills in the
# sd1:kids/sd2:kids LR/df/p columns instead of leaving them blank -- no
# separate rpbnb_tmb_boundary_tests() call needed. (Gaussian-copula evaluation
# runs multithreaded by default since 0.4.6 -- see NEWS.md -- so
# COPULA_FAMILY = "normal" needs no extra argument either.)
cat(sprintf(
  "=== Fitting RP-BNB (TMB engine, method = sml, %s copula, random kids) ===\n",
  COPULA_FAMILY))
t_copula <- system.time(
  fit_copula <- rpbnb(f1, f2, data = d, engine = "tmb", method = "sml",
                      random_1 = "kids",
                      random_2 = "kids",
                      dependence = copula(COPULA_FAMILY),
                      draws = DRAWS,
                      seed = SEED,
                      control = ctrl,
                      boundary_tests = LR_TEST)
)[["elapsed"]]
cat(sprintf("Estimation finished in %.1f s\n\n", t_copula))

# ---- Convergence gate before the boundary tests ------------------------------
# rpbnb_tmb_boundary_tests() refuses to run on a full model that nlminb did
# not drive to stationarity (code 0), so check it here and explain WHY rather
# than surfacing the cryptic stop from the boundary tester.
#
# "false convergence (8)" on THIS specification is a symptom of weak
# identification, not of the optimizer giving up too early: the score is far
# from stationarity AND the Hessian is not positive definite. Verified by a
```

```
# keep = "full" re-fit and eigendecomposition of the Hessian AT THE OPTIMUM
# (last.par.best, not the MakeADFun zero start): 3 negative eigenvalues
# (-58.8, -43.2, -24.7) out of 19 free parameters, i.e. the "optimum" is a
# saddle, not a maximum. The two smallest negative directions load almost
# entirely on log_sd1:kids (0.99) and log_sd2:kids (0.95) -- both random-
# coefficient scales curve the wrong way -- and the largest (-58.8) mixes
# log_m2 (0.85) with log_sd2:kids (0.26). The gradient is dominated by the
# dispersion/scale/dependence block (log_m1, log_m2, z_dep) rather than clean
# coefficient signal. kids is a 0/1 dummy in both equations, and a random
# coefficient on a dummy is weakly identified under a Gaussian copula (NB
# dispersion trades off against the random-coefficient scale) -- the exact
# caveat in this script's header. No optimizer knob (more restarts, a bigger
# iterlim, a tighter rel.tol) can converge to a saddle; it will burn hours and
# return the same code. See the "Fix" below for what does.
if (!identical(fit_copula$optimizer$convergence, 0L)) {
  stop(
    "The full model did not converge (nlminb code ", fit_copula$optimizer$convergence,
    ": ", fit_copula$optimizer$message,
    "; max|gradient| = ", signif(fit_copula$optimizer$max_abs_gradient, 4),
    " vs tolerance ", signif(fit_copula$optimizer$gradient_tolerance, 4),
    "), and the Hessian is not positive definite (sdreport$pdHess = ",
    fit_copula$sdreport$pdHess, "). This is the weak-identification signature ",
    "of a 0/1 dummy (kids) as the random coefficient in both equations under a ",
    "Gaussian copula -- not an optimizer-settings problem.\n",
    "Fix: put the random coefficient on a CONTINUOUS regressor (kids is 0/1 in ",
    "both equations, and every other regressor here is also 0/1 -- so a ",
    "continuous column must come from the data), or load kids in only ONE ",
    "equation, or switch to a dependence structure that identifies the scale ",
    "better. Once the score is near stationarity, the boundary tests below run ",
    "unconditionally.",
    call. = FALSE)
}
print(summary(fit_copula))

# ---- (Optional) Saddle-point diagnostic ----------------------------------
# Un-comment to reproduce the weak-identification verification above. It
# confirms, from the curvature itself, that a non-converged fit is a saddle
# (negative curvature in the random-scale block) rather than a stalled trust
# region. Costs one more full Hessian evaluation at the optimum (o$he()), which
# is expensive on this data (n = 3874 x 500 draws), so it is off by default.
#
# Three requirements:
#   (1) keep = "full" so the TMB objective o <- fit$obj is stored (the
#       default keep = "postfit" does not retain it -- o$he()/o$gr() then
#       error). Re-fit identically except for keep, or refit this one.
#   (2) Evaluate at o$env$last.par.best (the true optimum, objective =
#       -logLik), NOT o$par -- o$par is the all-zero MakeADFun START, where the
#       objective (~3e4 here) and gradient (hundreds-to-thousands everywhere)
#       are meaningless and would masquerade as "the whole model is bad".
#   (3) Symmetric eigen() for both the eigenvalues (saddle test) and the
#       eigenvector loadings (which parameters the negative directions live in).
#
# Expected on this weakly-identified spec: 3 negative eigenvalues out of 19
# (the two smallest loading ~entirely on log_sd1:kids / log_sd2:kids, the
# largest mixing log_m2 with log_sd2:kids), confirming a saddle, not a maximum.
# A healthy, well-identified fit instead shows all-positive eigenvalues and a
# small max|gradient| at last.par.best.
```

```
#
# Runs only when the fit did not converge (the case worth inspecting). To
# check a fit that DID converge (a sanity check that all eigenvalues are
# positive), drop the `if` guard and run the body unconditionally.
#
# if (!identical(fit_copula$optimizer$convergence, 0L)) {
#   if (is.null(fit_copula$obj))
#     stop("Refit with keep = \"full\" first: fit$obj (the TMB objective) is ",
#          "not stored under the default keep = \"postfit\".")
#   o <- fit_copula$obj
#   p <- o$env$last.par.best
#   par_names <- names(coef(fit_copula))
#   H <- o$he(p)
#   g <- as.numeric(as.matrix(o$gr(p)))
#   cat("objective at optimum:", o$fn(p), " (== -logLik = ",
#       -as.numeric(fit_copula$logLik), ")\n")
#   cat("max|gradient| at optimum:", max(abs(g)), "\n")
#   ev <- eigen(H, symmetric = TRUE)
#   cat("Hessian eigenvalues (ascending):\n"); print(round(ev$values, 3))
#   cat("negative eigenvalues:", sum(ev$values < 0), " of", length(ev$values),
#       "\n")
#   # Which parameters the negative (saddle) directions live in:
#   for (i in which(ev$values < 0)) {
#     v <- ev$vectors[, i]
#     top <- order(abs(v), decreasing = TRUE)[1:6]
#     cat(sprintf("\neigenvalue %s:\n", signif(ev$values[i], 4)))
#     print(cbind(name = par_names[top], load = v[top]))
#   }
# }

# ---- Threading diagnostics -----------------------------------------------
# fit$parallel$realized can come in below $requested for reasons that are easy
# to conflate: no OpenMP in this build (openmp: FALSE below -- affects every
# family, e.g. Apple clang with no libomp linked), fewer threads actually
# supported than requested (TMB::openmp(max = TRUE) returned less than
# max_threads), or -- Gaussian copula only, and only on a pre-0.4.6 install --
# the single-thread safety cap that used to apply unconditionally (see
# ?fit_rpbnb_tmb's `disable_parallel_gaussian`). Printed here because
# COPULA_FAMILY = "normal" above is exactly the case that cap used to affect.
cat(sprintf("Threads requested: %d, realized: %d\n",
            fit_copula$parallel$requested, fit_copula$parallel$realized))
print(rpbnb_build_info())

# ---- inst/example_rpbnb_tmb_laplace.R: TMB, method = "laplace", copula dependence (rwm1984.csv) ----
#!/usr/bin/env Rscript
# ============================================================================
# example_rpbnb_tmb_laplace.R -- random-parameter bivariate NB (RP-BNB), TMB
# engine, method = "laplace", copula dependence
#
# Estimates a random-parameter bivariate NB2 model for the two count outcomes
# in the German health-care utilization panel (rwm1984.csv) under copula
# dependence (COPULA_FAMILY below), via the TMB engine's Laplace-approximation
# estimator. Same data, dummy variables (see inst/example_bnb.R), formulas,
# and random coefficient (kids, in both equations) as
# inst/example_rpbnb_tmb_sml.R -- this one is the "laplace" side of that
# script's estimator choice; run the two side by side to compare the two
# approximations on the same model.
```

```
#
# The TMB engine has two estimators for the random-coefficient integral:
# "sml" (simulated maximum likelihood over Halton draws, the same integral
# the classic engine approximates, but with an exact automatic-differentiation
# gradient instead of a numerical one) and "laplace" (a sparse-Hessian
# approximation over one latent vector per observation that removes `draws`
# from the memory cost -- tape size scales with nrow(data) alone, see
# ?fit_rpbnb_tmb's `method` argument). This script is the "laplace" side of
# that choice. The two are DIFFERENT approximations to the same integral:
# they agree asymptotically but need not agree closely on any given dataset,
# so a Laplace fit is not a drop-in reproduction of an SML fit. summary()
# still reports AIC/BIC, but under "laplace" they are computed from an
# approximated marginal likelihood, so an AIC from a Laplace fit is not
# meaningful to compare against an AIC from an SML fit of the same model.
# "laplace" also supports "normal" and "lognormal" random coefficients only
# and requires at least one random coefficient -- `kids` is Normal in both
# equations, so both hold here.
#
# `kids` is loaded as random in BOTH equations, for consistency with
# inst/example_rpbnb_tmb_sml.R and to avoid identifying the random
# coefficient off only one margin's variation. Note this cuts against
# ?fit_rpbnb_tmb's own advice for the copula path specifically: `kids` is a
# 0/1 dummy in this data, and a random coefficient on a dummy regressor is
# weakly identified under copula dependence (NB dispersion trades off against
# the random-coefficient scale) -- kept anyway here so the two scripts'
# specifications line up; a continuous regressor would identify more cleanly.
#
# Run with:   Rscript inst/example_rpbnb_tmb_laplace.R
# or, after install:
#   Rscript -e 'source(system.file("example_rpbnb_tmb_laplace.R", package = "rpbnb"))'
# ============================================================================

#library(rpbnb)
devtools::load_all()

# ---- Load data -------------------------------------------------------------
d <- read.csv(system.file("extdata", "rwm1984.csv", package = "rpbnb", mustWork = TRUE))

cat("=== rwm1984.csv ===\n")
cat("Observations:", nrow(d), "\n")
cat("docvis  (doctor visits) : mean =", round(mean(d$docvis), 2),
    " var =", round(var(d$docvis), 2), "\n")
cat("hospvis (hospital visits): mean =", round(mean(d$hospvis), 2),
    " var =", round(var(d$hospvis), 2), "\n\n")

# ---- Dummy variables from the raw categorical columns ----------------------
# Same derivation as inst/example_bnb.R -- see that script for the full
# explanation. postHS: schooling beyond the lowest track (edlevel >= 2). MarM/
# MarF/SinM/SinF: sex x marital-status, four mutually exclusive dummies; SinM
# (single male) is left out of the formulas below as the reference category.
d$postHS <- as.integer(d$edlevel >= 2)
d$MarM <- as.integer(d$married == 1 & d$female == 0)  # married male
d$MarF <- as.integer(d$married == 1 & d$female == 1)  # married female
d$SinM <- as.integer(d$married == 0 & d$female == 0)  # single male (reference)
d$SinF <- as.integer(d$married == 0 & d$female == 1)  # single female

f1 <- docvis  ~ outwork + kids + postHS + MarM + MarF + SinF
```

```
f2 <- hospvis ~ outwork + kids + postHS + MarM + MarF + SinF

# `draws`/`seed` (fit_rpbnb_tmb()'s Halton-grid arguments) are left at their
# defaults here on purpose. Under method = "laplace" `draws` does NOT affect
# the likelihood, the TMB tape, or any chunking (tape size scales with
# nrow(data) alone); its only remaining job is sizing the Halton grid used by
# predict() and the marginal-effect functions for post-estimation averaging
# (the frozen Famoye admissible-lambda interval is derived from the random
# coefficients' support instead, independent of draws/seed under either
# estimator -- see ?fit_rpbnb_tmb's `draws` argument). This script calls
# neither predict() nor a marginal-effect function, so draws/seed have no
# observable effect on anything it prints; set them explicitly only once you
# add such a call and want its averaging grid reproducible or sized to taste.
# Which copula to fit. copula() accepts exactly these three families;
# everything downstream (labels, the reported native parameter, Kendall's
# tau) adapts to whichever is set here.
#   "frank"     -- theta on the whole real line, no tail dependence
#   "normal"    -- Gaussian, rho in (-1, 1)
#   "kimeldorf" -- Kimeldorf-Sampson (Clayton), theta > 0, lower-tail dependence
COPULA_FAMILY <- "normal"

# rpbnb_control()'s n_cores is the OpenMP thread count for both TMB estimators
# (see ?rpbnb_control); leave 1 core free for the rest of the system. TMB's
# se_method-equivalent is the `inference` argument to fit_rpbnb_tmb() (full
# covariance by default), not a control field -- control$se_method is a
# classic-engine-only knob and is ignored here.
N_CORES <- max(1L, parallel::detectCores() - 1L)

# Which boundary_tests group(s) to run -- see ?rpbnb's boundary_tests table.
# "sd" tests only the random-coefficient scales (sd1:kids, sd2:kids); add
# "dispersion" (or pass TRUE for both) to also test m1/m2, at the cost of two
# more restricted refits.
LR_TEST <- "sd"

# ---- TMB engine control ------------------------------------------------------
# One rpbnb_control() object drives every estimator; here only the thread
# fields matter for this fit (n_cores, max_threads). The SML-only memory
# knobs (max_workload, tape_chunks) are NOT set: under method = "laplace" the
# tape scales with nrow(data) alone, so there is no draw dimension to chunk
# and the weighted-workload guard bounds nothing -- .resolve_tape_chunks()
# reports them as ignored rather than erroring (see ?rpbnb_control).
ctrl <- rpbnb_control(
  n_cores = N_CORES,
  # OpenMP thread count for both TMB estimators (see ?rpbnb_control). Left 1
  # core free above via N_CORES.
  max_threads = N_CORES,
  # Ceiling on OpenMP threads permitted for one TMB fit. NULL would default to
  # n_cores; set equal here so the ceiling never binds below the request.
  # (max_workload / tape_chunks deliberately omitted: SML-only, ignored for
  # method = "laplace".)
)
cat("Using", N_CORES, "OpenMP threads.\n\n")

# ---- Copula RP-BNB, TMB engine, Laplace --------------------------------------
# method = "laplace" uses TMB's Laplace approximation with a sparse Hessian
# over one latent vector per observation; tape size scales with nrow(data)
# alone, so this fit uses far less peak memory than the SML script's.
```

```
# boundary_tests = LR_TEST runs rpbnb_tmb_boundary_tests() on the converged fit
# and attaches it as fit$boundary_tests, so summary() below fills in the
# sd1:kids/sd2:kids LR/df/p columns instead of leaving them blank -- no
# separate rpbnb_tmb_boundary_tests() call needed. (Gaussian-copula evaluation
# runs multithreaded by default since 0.4.6 -- see NEWS.md -- so
# COPULA_FAMILY = "normal" needs no extra argument either.)
cat(sprintf(
  "=== Fitting RP-BNB (TMB engine, method = laplace, %s copula, random kids) ===\n",
  COPULA_FAMILY))
t_copula <- system.time(
  fit_copula <- rpbnb(f1, f2, data = d, engine = "tmb", method = "laplace",
                      random_1 = "kids",
                      random_2 = "kids",
                      dependence = copula(COPULA_FAMILY),
                      control = ctrl,
                      boundary_tests = LR_TEST)
)[["elapsed"]]
cat(sprintf("Estimation finished in %.1f s\n\n", t_copula))
print(summary(fit_copula))

# ---- Threading diagnostics -------------------------------------------------
# fit$parallel$realized can come in below $requested for reasons that are easy
# to conflate: no OpenMP in this build (openmp: FALSE below -- affects every
# family, e.g. Apple clang with no libomp linked), fewer threads actually
# supported than requested (TMB::openmp(max = TRUE) returned less than
# max_threads), or -- Gaussian copula only, and only on a pre-0.4.6 install --
# the single-thread safety cap that used to apply unconditionally (see
# ?fit_rpbnb_tmb's `disable_parallel_gaussian`) -- directly relevant here
# since COPULA_FAMILY = "normal" above is exactly the case that cap affected.
cat(sprintf("Threads requested: %d, realized: %d\n",
            fit_copula$parallel$requested, fit_copula$parallel$realized))
print(rpbnb_build_info())

# ---- inst/example_rpbnb_dense_tmb_sml.R: TMB, method = "sml", dense-section truck-crash data (local research
#!/usr/bin/env Rscript
# ============================================================================
# rpbnb(engine = "tmb", method = "sml") on the dense-section truck-crash data.
#
# A trimmed version of inst/dev/rpbnb_truck_dense.R: that script fits BOTH
# engines from one rpbnb_control() object to demonstrate the control object is
# a superset either engine accepts. This script keeps the same dense model
# structure but fits only the TMB engine, so there is no engine loop, no
# engine-comparison section, and no maxLik-only control knobs (se_method,
# hess_eps) -- nothing here reads them. It is the dense-data counterpart of
# inst/dev/rpbnb_truck_open_v2.R; see that header for the same trimming
# rationale.
#
# DATA and MODEL STRUCTURE (taken from rpbnb_truck_dense.R):
#
#   * inst/extdata/export_dense_all.csv (dense sections), not the
#     open-section export_open_all.csv.
#   * The two equations use DIFFERENT covariate sets -- ALL_3 in eq 1,
#     C_DISTR in eq 2 -- and one random coefficient on SR40_MI3 in each
#     equation. The same random coefficient, weakly identified, is shared as
#     in the open scripts so every truck script fits the same style of model.
#   * dependence = "famoye" (Famoye/Sarmanov), matching the referenced dense
#     script, rather than a copula.
```

```
#
# standardize = TRUE (same as rpbnb_truck_dense.R): the dense data carries
# strictly-positive continuous covariates (SR40_MI3, MPD_ME, MPD_STD, ...)
# that would otherwise make a random-coefficient carrier a random intercept in
# disguise and would inflate the design-matrix condition number. summary()
# back-transforms the coefficient table to original units automatically.
#
# COST. One full fit plus one restricted refit per tested parameter: 2
# random-coefficient scales (SR40_MI3 in each equation) + 2 dispersions +
# 1 dependence = 5 refits, i.e. roughly six full fits total. Drop
# boundary_tests to c("dispersion", "dependence") for a much cheaper run, or
# set max_rows below for a smoke test.
#
# devtools::load_all() (not library()): the script must run against the
# current source tree. Run from the package root:
#     Rscript inst/example_rpbnb_dense_tmb_sml.R
# ============================================================================

# library(rpbnb)  # use this instead of load_all() once installed
devtools::load_all()

# Prints a horizontal separator line between output sections.
sep <- function() cat("\n", paste(rep("=", 72), collapse = ""), "\n", sep = "")

# ---- Knobs ------------------------------------------------------------------
# n_cores: OpenMP threads the TMB engine uses for the simulated-likelihood
#          draws. This is the ONE parallel knob the TMB engine reads; max_threads
#          (a cap on it, defaulting to n_cores) is not set here because
#          requesting n_cores and capping at n_cores is the same thing.
n_cores <- 12L

# draws: number of Halton simulation draws. Under method = "sml" these
#        define the simulated likelihood being maximized.
draws   <- 1000L

# seed: drives the Cranley-Patterson rotation of the Halton draws, making the
#       draw grid (and so the fit) reproducible run-to-run.
seed    <- 20240712L

# tmb_method: the TMB engine's estimator. "sml" (simulated ML) or
#             "laplace" (TMB's memory-saving analytic alternative).
tmb_method <- "sml"

# dependence: the inter-equation dependence. "famoye" (Famoye/Sarmanov) here.
#             A copula() object is also accepted, one of three families:
#               copula("frank")       -- Frank copula (theta; symmetric)
#               copula("normal")      -- Gaussian copula (rho in (-1, 1))
#               copula("kimeldorf")   -- Clayton copula (theta > 0)
#             (an optional par argument fixes the dependence parameter for
#             simulation). Note copula("normal") runs multithreaded by default
#             (since 0.4.6 -- the SIGSEGV the old single-thread cap guarded
#             against was fixed in 0.4.4), so set disable_parallel_gaussian =
#             TRUE on the fit to pin a Gaussian run to one thread.
dependence <- copula("normal")

# boundary_tests: which parameter groups to LR-test against their boundary
#                 restriction. "all" = c("sd", "dispersion", "dependence") --
```

```
#                 one restricted refit per tested parameter. FALSE or "none"
#                 skips the LR tests entirely.
boundary_tests <- FALSE

# boundary_draws: draws used by the restricted LR refits. NULL reuses the main
#                 fit's `draws`, which keeps the restricted and full simulated
#                 likelihoods on common random numbers -- diverge only
#                 deliberately.
boundary_draws <- NULL

# max_rows: smoke-test cap. NULL uses every row; set e.g. 400L to rehearse the
#           whole script cheaply before committing to a full run.
max_rows <- NULL

data <- read.csv(system.file("extdata", "export_dense_all.csv", package = "rpbnb", mustWork = TRUE))
if (!is.null(max_rows)) {
  data <- utils::head(data, max_rows)
  cat("*** SMOKE TEST: using the first", max_rows, "rows only ***\n")
}

# is_cop / dep_label: build the header label from `dependence` -- a copula
# object reports its family name, a bare string ("famoye") reports itself.
is_cop <- inherits(dependence, "rpbnb_copula")
dep_label <- if (is_cop) paste0(dependence$family, " copula") else as.character(dependence)
cat("=== rpbnb(engine = \"tmb\") on truck all crashes, dense sections (",
    dep_label, ") ===\n", sep = "")
cat("Observations   :", nrow(data), "\n")
cat("Cores asked    :", n_cores, "\n")
cat("Draws          :", draws, "\n")
cat("LR test groups :",
    if (isFALSE(boundary_tests)) "none" else paste(boundary_tests, collapse = ", "),
    "\n")

# f1, f2: the two marginal equations. Each is a count response regressed on a
# covariate set; the equations deliberately use DIFFERENT covariate sets
# (ALL_3 in eq 1, C_DISTR in eq 2), taken from rpbnb_truck_dense.R.
f1 <- ALL_3  ~ LNAADT_3 + SR40_MI3 + MPD_ME + MPD_STD + IRI_ME + RUT_L + SP50GE + ACCPNTS + SIGNAL1 + NEAR_SIG + A
f2 <- C_DISTR ~ LNAADT_3 + SR40_MI3 + MPD_ME + MPD_STD + RUT_9 + ACCPNTS + SIGNAL1 + NEAR_SIG + AUXLNUM + DP01_ME

cat("Equation 1     :", deparse(f1), "\n")
cat("Equation 2     :", deparse(f2), "\n")

# random_1, random_2: the covariates given a random (varying) slope in each
# equation, i.e. whose coefficient varies across observations from a random
# distribution. Both equations put the one random slope on SR40_MI3.
random_1 <- c("SR40_MI3")
random_2 <- c("SR40_MI3")

# ---- Control object (TMB knobs only) ----------------------------------------
# Three TMB settings are named here; only one of them differs from its package
# default:
#   max_workload -- the TMB workload guard's budget. Set to Inf to disable the
#                   guard: its default calibration under-estimates this data's
#                   per-draw cost (see rpbnb_truck_dense.R's header), so leaving
#                   it engaged would block the fit rather than merely warn.
#   print_level  -- verbosity, and the switch that silences the per-refit
#                   LR-test messages. Kept at the TMB default of 1 but stated
```

```
#                   so that the per-refit LR-test messages it gates are
#                   explained.
#   n_cores      -- OpenMP threads for the simulated-likelihood draws; the
#                   single parallel knob the TMB engine reads.
# Everything else the TMB engine reads -- iterlim, reltol, halton_burn,
# gradtol, restarts, max_threads -- is left at its package default and
# therefore not named here (reltol resolves to 1e-8, its default).
ctrl <- rpbnb_control(
  print_level  = 1,
  n_cores      = n_cores,
  parallel_tape = TRUE
)

# The banner carries the same engine/method/dependence/draws identity the
# print below is narrowed to, so the header names the parameters it is about
# to show (the print itself has no dependence field to echo).
sep(); cat(sprintf(
  "CONTROL OBJECT (engine = \"tmb\", method = \"%s\", dependence = \"%s\", draws = %d)\n",
  tmb_method, dep_label, draws)); sep()
# engine=/method=/draws narrow this print to the settings the TMB + SML fit
# actually reads and report the simulated-likelihood draw count.
print(ctrl, engine = "tmb", method = tmb_method, draws = draws)

# ---- Fit ------------------------------------------------------------------
# stamp / results: a timestamped run label and the directory the fit is saved
# to.
stamp <- format(Sys.time(), "%Y-%m-%d-%H%M%S")
dir.create("results", recursive = TRUE, showWarnings = FALSE)

sep(); cat("FIT: engine = \"tmb\"\n", sep = ""); sep()

# t_fit: wall-clock seconds for the whole fit, INCLUDING the restricted LR
# refits boundary_tests triggers.
t_fit <- system.time(
  fit <- rpbnb(
    formula_1      = f1,
    formula_2      = f2,
    data           = data,
    engine         = "tmb",
    method         = tmb_method,
    boundary_draws = boundary_draws,
    random_1       = random_1,
    random_2       = random_2,
    dependence     = dependence,
    seed           = seed,
    draws          = draws,
    standardize    = TRUE,
    boundary_tests = boundary_tests,
    control        = ctrl
  )
)[["elapsed"]]

cat(sprintf("\nFinished in %.2f s%s\n", t_fit,
            if (isFALSE(boundary_tests) || !length(boundary_tests)) ""
            else " (includes the restricted LR refits)"))
# Convergence of the TMB engine's nlminb optimizer (code + message).
cat(sprintf("Convergence  : nlminb code=%d, %s\n",
```

```
            fit$optimizer$convergence, fit$optimizer$message))

fit_path <- file.path("results", paste0("fit_truck_dense_tmb_", stamp, ".rds"))
saveRDS(fit, fit_path)
cat("Saved to     :", fit_path, "\n")

# Names any supplied control setting the TMB engine did not read (none are
# expected here, since ctrl sets only TMB-applicable fields).
cat("Control settings this engine did not read: ",
    if (length(fit$control_ignored)) paste(fit$control_ignored, collapse = ", ")
    else "(none)", "\n", sep = "")

# ---- Summary -------------------------------------------------------------
# standardize = TRUE back-transforms the coefficient table to original units
# automatically. With the LR tests attached, the natural-scale block carries
# LR/df/p for the scale and dispersion rows in place of the NA they would
# otherwise show, and the dependence row shows its LR test instead of a Wald z.
sep(); cat("MODEL SUMMARY (original covariate units)\n"); sep()
summary(fit)
cat("\n")

# ---- LR tests, standalone ------------------------------------------------
sep(); cat("LR TESTS\n"); sep()
if (is.null(fit$boundary_tests)) {
  cat("Skipped: `boundary_tests` named no group.  Set it at the top of this\n")
  cat("script -- \"all\", or e.g. c(\"dispersion\", \"dependence\") for the\n")
  cat("cheap subset -- to run them.\n")
} else {
  print(fit$boundary_tests)
}

sep(); cat("DONE\n"); sep()

# ---- inst/example_rpbnb_dense_tmb_laplace.R: TMB, method = "laplace", dense-section truck-crash data (local
#!/usr/bin/env Rscript
# ============================================================================
# rpbnb(engine = "tmb", method = "laplace") on the dense-section truck-crash data.
#
# The method = "laplace" twin of inst/example_rpbnb_dense_tmb_sml.R: the same
# dense model structure (same data, equations, and random coefficient) run
# through the TMB engine's Laplace approximation instead of its SML estimator.
# Laplace is the TMB engine's memory-saving analytic alternative to SML: it
# works over one latent vector per observation, so the tape -- and peak
# memory -- scale with nrow(data) alone rather than nrow(data) * draws (see
# ?fit_rpbnb_tmb's `method` argument). The two estimators approximate the same
# random-coefficient integral DIFFERENTLY; they agree asymptotically but need
# not agree closely on any given dataset, so this is not a drop-in
# reproduction of the SML script's fit.
#
# Under method = "laplace" the SML-only parameters have no effect and are
# therefore not set anywhere in this script:
#   * `draws` / `seed` -- the Halton draw grid. Under Laplace `draws` does not
#     affect the likelihood, the tape, or any chunking; its only remaining job
#     is sizing the averaging grid for predict() and the marginal-effect
#     functions. This script calls none of those, so draws/seed are left at
#     their package defaults.
#   * `boundary_draws` -- the draws for the restricted LR refits. SML-only for
```

```
#     the same reason; the LR refits here are Laplace fits too, so there is no
#     draw dimension to keep on common random numbers.
#
# DATA and MODEL STRUCTURE (taken from rpbnb_truck_dense.R, same as the SML
# twin):
#
#   * inst/extdata/export_dense_all.csv (dense sections), not the
#     open-section export_open_all.csv.
#   * The two equations use DIFFERENT covariate sets -- ALL_3 in eq 1,
#     C_DISTR in eq 2 -- and one random coefficient on SR40_MI3 in each
#     equation. The same random coefficient, weakly identified, is shared as
#     in the open scripts so every truck script fits the same style of model.
#   * dependence = "famoye" (Famoye/Sarmanov), matching the referenced dense
#     script, rather than a copula.
#
# standardize = TRUE (same as rpbnb_truck_dense.R): the dense data carries
# strictly-positive continuous covariates (SR40_MI3, MPD_ME, MPD_STD, ...)
# that would otherwise make a random-coefficient carrier a random intercept in
# disguise and would inflate the design-matrix condition number. summary()
# back-transforms the coefficient table to original units automatically.
#
# COST. One full fit plus one restricted refit per tested parameter: 2
# random-coefficient scales (SR40_MI3 in each equation) + 2 dispersions +
# 1 dependence = 5 refits, i.e. roughly six full fits total. Drop
# boundary_tests to c("dispersion", "dependence") for a much cheaper run, or
# set max_rows below for a smoke test.
#
# devtools::load_all() (not library()): the script must run against the
# current source tree. Run from the package root:
#     Rscript inst/example_rpbnb_dense_tmb_laplace.R
# ============================================================================

# library(rpbnb)  # use this instead of load_all() once installed
devtools::load_all()

# Prints a horizontal separator line between output sections.
sep <- function() cat("\n", paste(rep("=", 72), collapse = ""), "\n", sep = "")

# ---- Knobs ------------------------------------------------------------------
# n_cores: OpenMP threads the TMB engine uses. This is the ONE parallel knob
#          the TMB engine reads; max_threads (a cap on it, defaulting to
#          n_cores) is not set here because requesting n_cores and capping at
#          n_cores is the same thing.
n_cores <- 23L

# tmb_method: the TMB engine's estimator. "laplace" here -- the memory-saving
#             Laplace approximation, as opposed to "sml" (the SML twin's
#             estimator), which maximizes the simulated likelihood over
#             Halton draws.
tmb_method <- "laplace"

# dependence: the inter-equation dependence. "famoye" (Famoye/Sarmanov) here.
#             A copula() object is also accepted, one of three families:
#               copula("frank")       -- Frank copula (theta; symmetric)
#               copula("normal")      -- Gaussian copula (rho in (-1, 1))
#               copula("kimeldorf")   -- Clayton copula (theta > 0)
#             (an optional par argument fixes the dependence parameter for
```

```
#             simulation). Note copula("normal") runs multithreaded by default
#             (since 0.4.6 -- the SIGSEGV the old single-thread cap guarded
#             against was fixed in 0.4.4), so set disable_parallel_gaussian =
#             TRUE on the fit to pin a Gaussian run to one thread.
dependence <- copula("frank")

# boundary_tests: which parameter groups to LR-test against their boundary
#                 restriction. "all" = c("sd", "dispersion", "dependence") --
#                 one restricted refit per tested parameter. FALSE or "none"
#                 skips the LR tests entirely.
boundary_tests <- FALSE

# max_rows: smoke-test cap. NULL uses every row; set e.g. 400L to rehearse the
#           whole script cheaply before committing to a full run.
max_rows <- NULL

data <- read.csv(system.file("extdata", "export_dense_all.csv", package = "rpbnb", mustWork = TRUE))
if (!is.null(max_rows)) {
  data <- utils::head(data, max_rows)
  cat("*** SMOKE TEST: using the first", max_rows, "rows only ***\n")
}

# is_cop / dep_label: build the header label from `dependence` -- a copula
# object reports its family name, a bare string ("famoye") reports itself.
is_cop <- inherits(dependence, "rpbnb_copula")
dep_label <- if (is_cop) paste0(dependence$family, " copula") else as.character(dependence)
cat("=== rpbnb(engine = \"tmb\", method = \"laplace\") on truck all crashes, dense sections (",
    dep_label, ") ===\n", sep = "")
cat("Observations   :", nrow(data), "\n")
cat("Cores asked    :", n_cores, "\n")
cat("LR test groups :",
    if (isFALSE(boundary_tests)) "none" else paste(boundary_tests, collapse = ", "),
    "\n")

# f1, f2: the two marginal equations. Each is a count response regressed on a
# covariate set; the equations deliberately use DIFFERENT covariate sets
# (ALL_3 in eq 1, C_DISTR in eq 2), taken from rpbnb_truck_dense.R.
f1 <- ALL_3  ~ LNAADT_3 + SR40_MI3 + MPD_ME + MPD_STD + IRI_ME + RUT_L + SP50GE + ACCPNTS + SIGNAL1 + NEAR_SIG + A
f2 <- C_DISTR ~ LNAADT_3 + SR40_MI3 + MPD_ME + MPD_STD + RUT_9 + ACCPNTS + SIGNAL1 + NEAR_SIG + AUXLNUM + DP01_ME

cat("Equation 1     :", deparse(f1), "\n")
cat("Equation 2     :", deparse(f2), "\n")

# random_1, random_2: the covariates given a random (varying) slope in each
# equation, i.e. whose coefficient varies across observations from a random
# distribution. Both equations put the one random slope on SR40_MI3.
random_1 <- c("SR40_MI3")
random_2 <- c("SR40_MI3")

# ---- Control object (TMB knobs only) ----------------------------------------
# One TMB setting is named here -- n_cores, the OpenMP thread count for the
# Laplace fit; the single parallel knob the TMB engine reads. print_level is
# left at the TMB default of 1 (it is also the switch that silences the
# per-refit LR-test messages, none of which fire while boundary_tests is
# FALSE). Everything else the TMB engine reads -- iterlim, reltol, gradtol,
# restarts, max_threads, parallel_tape -- is left at its package default and
# therefore not named here. The SML-only memory knobs (max_workload,
```

```
# tape_chunks) are likewise omitted: under method = "laplace" the tape scales
# with nrow(data) alone, so there is no draw dimension to chunk and the
# weighted-workload guard bounds nothing.
ctrl <- rpbnb_control(
  n_cores = n_cores
)

# The banner carries the same engine/method/dependence identity the print
# below is narrowed to, so the header names the parameters it is about to show
# (the print itself has no dependence field to echo).
sep(); cat(sprintf(
  "CONTROL OBJECT (engine = \"tmb\", method = \"%s\", dependence = \"%s\")\n",
  tmb_method, dep_label)); sep()
# engine=/method= narrow this print to the settings the TMB + Laplace fit
# actually reads.
print(ctrl, engine = "tmb", method = tmb_method)

# ---- Fit ---------------------------------------------------------------
# stamp / results: a timestamped run label and the directory the fit is saved
# to.
stamp <- format(Sys.time(), "%Y-%m-%d-%H%M%S")
dir.create("results", recursive = TRUE, showWarnings = FALSE)

sep(); cat("FIT: engine = \"tmb\", method = \"laplace\"\n", sep = ""); sep()

# t_fit: wall-clock seconds for the whole fit, INCLUDING the restricted LR
# refits boundary_tests triggers.
t_fit <- system.time(
  fit <- rpbnb(
    formula_1      = f1,
    formula_2      = f2,
    data           = data,
    engine         = "tmb",
    method         = tmb_method,
    random_1       = random_1,
    random_2       = random_2,
    dependence     = dependence,
    standardize    = TRUE,
    boundary_tests = boundary_tests,
    control        = ctrl
  )
)[["elapsed"]]

cat(sprintf("\nFinished in %.2f s%s\n", t_fit,
            if (isFALSE(boundary_tests) || !length(boundary_tests)) ""
            else " (includes the restricted LR refits)"))
# Convergence of the TMB engine's nlminb optimizer (code + message).
cat(sprintf("Convergence  : nlminb code=%d, %s\n",
            fit$optimizer$convergence, fit$optimizer$message))

fit_path <- file.path("results", paste0("fit_truck_dense_tmb_", stamp, ".rds"))
saveRDS(fit, fit_path)
cat("Saved to     :", fit_path, "\n")

# Names any supplied control setting the TMB engine did not read (none are
# expected here, since ctrl sets only TMB-applicable fields).
cat("Control settings this engine did not read: ",
```

```
    if (length(fit$control_ignored)) paste(fit$control_ignored, collapse = ", ")
    else "(none)", "\n", sep = "")

# ---- Summary -------------------------------------------------------------
# standardize = TRUE back-transforms the coefficient table to original units
# automatically. With the LR tests attached, the natural-scale block carries
# LR/df/p for the scale and dispersion rows in place of the NA they would
# otherwise show, and the dependence row shows its LR test instead of a Wald z.
sep(); cat("MODEL SUMMARY (original covariate units)\n"); sep()
summary(fit)
cat("\n")

# ---- LR tests, standalone ------------------------------------------------
sep(); cat("LR TESTS\n"); sep()
if (is.null(fit$boundary_tests)) {
  cat("Skipped: `boundary_tests` named no group.  Set it at the top of this\n")
  cat("script -- \"all\", or e.g. c(\"dispersion\", \"dependence\") for the\n")
  cat("cheap subset -- to run them.\n")
} else {
  print(fit$boundary_tests)
}

sep(); cat("DONE\n"); sep()

## End(Not run)
```

---

`rpbnb_boundary_tests` *Boundary-corrected LR tests for all boundary parameters of an rpbnb_fit*

---

## Description

Runs a likelihood-ratio test for every random-coefficient standard deviation (`sd1:*`, `sd2:*`) and NB2 dispersion (`m1`, `m2`) of a fitted random-parameter bivariate NB model, and merges them into one table. These are the parameters whose null lies on the boundary of the parameter space (SD = 0, or dispersion `m` = `0` = Poisson), for which the natural-scale summary reports no Wald z/p; each test uses the 50:50 chi-square boundary correction of `lr_test()` (`boundary` = `TRUE`).

## Usage

```
rpbnb_boundary_tests(
  fit,
  data,
  control = rpbnb_control(compute_se = FALSE),
  which = c("sd", "dispersion")
)
```

## Arguments

| | |
|---|---|
| `fit` | A converged `rpbnb_fit` (the full model), from a Famoye or a `copula()` dependence. Both paths use the exact `m` = `0` branch for the dispersion tests. |
| `data` | The data frame the model was fit on. Required – the fit object does not store it, and every restricted model is refit on the same data. |

`control` An `rpbnb_control()` for the restricted refits. Defaults to `compute_se = FALSE` (the LR test needs only `logLik` and the degrees of freedom). `draws`/`draw_type`/`seed` are taken from `fit`, not `control`.

`which` Which parameter groups to test, any subset of `"sd"` (the random-coefficient scales), `"dispersion"` (the NB2 dispersions `m1`, `m2`), and `"dependence"` (the association parameter). The default is `c("sd", "dispersion")` – the two boundary-null groups. `"dependence"` is opt-in because it costs another full refit and, for three of the four families, tests an *interior* null whose ordinary Wald `z` in `summary()` is already valid.

The dependence test restricts the model to independence and compares it to `fit`: one row labelled `lam` (Famoye), `theta` (Frank / Clayton), or `rho` (Gaussian). The restriction pins the working-scale dependence parameter at its family's independence value rather than refitting a different family, so both fits keep the same draws and the same parameter block and the comparison is a clean 1-df restriction. Only Clayton's null (`theta > 0`, so `theta = 0` is a boundary) takes the 50:50 mixture; Famoye, Frank, and Gaussian nulls are interior and get an ordinary chi-square(1).

## Details

Each test refits a properly nested restricted model. With **multiple random coefficients in an equation, each SD is tested individually** (a 1-df restriction). The restricted fit keeps the full random specification but sets the tested coefficient's draw column to the distribution median (`u = 0.5`), which zeroes that coefficient's per-draw *deviation* exactly for every supported distribution (`u = 0.5` maps to base 0 for normal/lognormal/ triangular, and to the centered value `2 * 0.5 - 1 = 0` for uniform). The coefficient therefore collapses to its SD-zero null – the ordinary fixed coefficient `b` for normal/uniform/triangular, and `sign * exp(b)` for lognormal – on every draw, independent of the covariate scale. Its (now inert) log-scale is pinned only to drop it from the free-parameter count. Every other draw column is the full model's exact stored draw, so the two simulated log-likelihoods are compared on common random numbers, and each restricted fit is **warm-started from the full fit's coefficients** so the start-sensitive simulated objective does not settle at an inferior optimum. A restricted fit that fails to converge yields `NA` inference (with a warning) rather than a p-value, and a non-converged full `fit` is rejected.

## Value

An object of class `rpbnb_boundary_tests`: a data frame with columns `Parameter`, `LR`, `df`, `p.value`, `Signif` (one row per boundary parameter), and a `print` method.

Under Famoye dependence with uniform or triangular random coefficients (or a single varying margin), the full and restricted fits' admissible lambda intervals are frozen at different starting values, so each LR statistic compares maxima over slightly different lambda ranges; see the "Famoye caveat" section of `lr_test()`. Normal/lognormal coefficients in both margins are unaffected (the interval is the constant `c(-1, 1)`).

## See Also

`lr_test()`, `fit_rpbnb()`

## Examples

```
sim <- simulate_rpbnb(n = 600,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
```

```
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 200, seed = 1)
rpbnb_boundary_tests(fit, sim$data)
```

---

`rpbnb_build_info` *Report how this installation of rpbnb was compiled*

---

## Description

Nearly all of this package's running time is compiled likelihood evaluation, so whether its shared object was built with optimization is a performance fact worth roughly a factor of two on every fit – and nothing in R's own output tells you which build you have. This reports what the compiler actually did.

## Usage

```
rpbnb_build_info()
```

## Details

A source install (`install.packages(type = "source")`, `R CMD INSTALL`, `remotes::install_github()`) compiles with R's own `CXXFLAGS`, which is `-O2` on every standard platform, so the optimized build is what you get by default. The package does not – and by CRAN policy must not – override those flags. Two things do produce a slow build: a `-O0` (or `-Og`) entry in the user's `~/.R/Makevars`, which applies to every package compiled on that machine, and development helpers that inject debug flags of their own, notably `pkgbuild::compile_dll(debug = TRUE)` – its default – which `devtools::load_all()` uses when recompiling changed sources.

## Value

A list, invisibly when printed:

`optimized` `TRUE` when compiled at `-O1` or above, read from the compiler's own `__OPTIMIZE__` macro rather than inferred.

`openmp` `TRUE` when OpenMP is available, so `n_cores > 1` can do anything. A build without it fits single-threaded whatever `control$n_cores` says.

`openmp_max_threads` The OpenMP runtime's own thread ceiling.

`assertions_enabled` `TRUE` when `NDEBUG` is unset, a further marker of a debug build.

`compiler` Compiler and version that built the shared object.

## Examples

```
rpbnb_build_info()
```

rpbnb_control *Control parameters for every rpbnb estimator*

## Description

One control object for all of the package's fitters – `fit_bnb()`, `fit_rpbnb()`, `fit_rpbnb_tmb()`, and `rpbnb()` with either `engine`. It carries the union of the tuning knobs the estimators use; each fitter reads the ones that apply to it and **ignores the rest**, reporting the ignored names in `print()`/`summary()` of the resulting fit rather than erroring. That is what makes a script able to flip `engine = "classic"` to `"tmb"` without rewriting its control call.

## Usage

```
rpbnb_control(
  method = c("BFGS"),
  iterlim = NULL,
  reltol = 1e-08,
  print_level = NULL,
  draws_hessian = 100L,
  halton_burn = 300L,
  n_cores = 1L,
  compute_se = TRUE,
  hessian = c("numeric", "analytic"),
  se_method = NULL,
  hess_eps = 1e-05,
  hess_r = 4L,
  gradtol = 1e-05,
  restarts = 10L,
  max_threads = NULL,
  max_workload = NULL,
  parallel_tape = TRUE,
  tape_chunks = NULL
)
```

## Arguments

| | |
|---|---|
| `method` | Optimizer used by the `maxLik` fitters. Only "BFGS" is implemented and wired through; it is the sole accepted value. |
| `iterlim` | Maximum optimizer iterations. `NULL` (default) uses 300 for the `maxLik` fitters and 500 for the TMB engine's `nlminb`. |
| `reltol` | Relative convergence tolerance. |
| `print_level` | Optimizer verbosity, and the switch that silences the boundary-test progress messages. `NULL` (default) uses 2 under `maxLik` and 1 under the TMB engine. Under TMB it drives two separate switches – `MakeADFun(silent = print_level == 0)` and `nlminb`'s `trace = max(0, print_level - 1)` – so the levels are:<br>`0` Silent (the pre-0.4.6 TMB default).<br>`1` TMB's own output only: an `outer mgc:` line per outer evaluation, plus the one-time tape/atomic construction on the first fit of a session. No `nlminb` trace. |

2 Adds nlminb's per-iteration objective and parameter vector – the lowest level that traces every iteration.

>2 nlminb's trace is a print *interval*, not a verbosity level, so higher values print the objective *less* often.

Note that rpbnb_tmb_boundary_tests() builds its own control when one is not supplied, so its restricted refits print at their own default regardless of this setting.

draws_hessian Retained for backward compatibility but unused by every estimator: the random-parameter numeric Hessian is taken with the same optimization draws that produced the estimate (same-draw curvature), so it no longer resimulates a separate Hessian draw set. Supplying it is always reported as an ignored setting.

halton_burn Number of leading Halton points discarded before forming the simulation draws.

n_cores Worker processes for fit_rpbnb()'s optional cluster path, or OpenMP threads for fit_rpbnb_tmb() (1 = sequential in both cases). Under the TMB engine the *realized* thread count – after max_threads and hardware capping – is also what max_workload/tape_chunks size the tape against when parallel_tape = TRUE; see "How n_cores, max_workload, and tape_chunks interact" below.

compute_se If FALSE, skip the Hessian and standard errors. The TMB engine has its own inference argument for this instead.

hessian How fit_bnb() (famoye) computes the Hessian for standard errors: "numeric" (default, numDeriv::hessian()) or "analytic" (the closed-form Famoye (2010) Appendix Hessian). Both freeze the lambda-bounds at the optimum and yield the same observed-information SEs.

se_method Standard-error method for fit_rpbnb() (the random-parameter model): "numeric" uses the numDeriv::hessian() observed-information Hessian; "analytic" uses the closed-form observed-information Hessian (Famoye (2010) per-draw second derivatives assembled via the Louis mixture formula) – exact and much faster than "numeric" for larger models; "opg" uses the BHHH / outer-product-of-gradients information from the per-observation scores – fastest (one gradient pass, versus "numeric"'s O(npar^2) finite-difference log-likelihood evaluations), but relies on the information-matrix equality so it is unreliable for parameters at a boundary (e.g. a random-coefficient SD estimated near 0). For copula dependence (fit_rpbnb() with dependence = copula(...)), only "opg" and "numeric" are available; "analytic" is not implemented for the copula path and errors.

Default NULL, meaning "this dependence family's own default", resolved once the fit is dispatched (same NULL-until-resolved pattern as iterlim/print_level): "numeric" for dependence = "famoye", "opg" for dependence = copula(...). The two differ because their speed/reliability tradeoffs differ – Famoye's fast, exact "analytic" Hessian makes "numeric" (not "opg") the conservative default there, where the copula path has no analytic Hessian at all and its own numeric Hessian is markedly slower than OPG (measured ~3x on an 11-parameter fit) for SEs that mostly agree with it anyway, except (per the boundary caveat above) on random-coefficient scale parameters – fall back to "numeric" for just those, or use rpbnb_boundary_tests(), if precise inference on an SD estimated near 0 matters. Read back a resolved fit's own fit$control$se_method (or the value read off object$control reported by print()/summary()) rather than assuming which default applied.

hess_eps, hess_r

Step and Richardson order for numDeriv::hessian() (used only when the numeric Hessian is selected).

`gradtol` Stationarity tolerance for the TMB engine's score, applied *relative* to the objective: the fit is treated as stationary once `max(abs(gradient))` falls below `gradtol * max(1, abs(nll))`. `nlminb()` declares convergence from its relative function test, which can fire while the score is still far from zero; a Hessian taken there is not a curvature and its inverse can carry negative variances. The fit is restarted until the gradient clears this tolerance; missing it is not warned about on its own, because the copula families legitimately optimise against a clamp or a probability floor where no step improves the objective, but it is reported in the warning raised when the Hessian actually fails to factor, and is kept on the fit as `optimizer$max_abs_gradient`. The scaling matters because the score is a sum over observations, so an absolute cutoff would tighten with sample size for no statistical reason.

`restarts` Maximum number of times to restart `nlminb()` from its own answer while `gradtol` is unmet. Restarting resets the trust region and step scaling that stalled the first solve. Set to `0L` for the single-call behaviour of earlier versions.

`max_threads` Maximum OpenMP threads permitted for one TMB fit. `NULL` (default) means `n_cores`, so threads are not capped unless set explicitly below `n_cores`.

`max_workload` Maximum weighted observation-draw evaluations permitted before TMB tape construction; `Inf` disables the guard. The default and every figure here are derived from `TAPE_CALIBRATION`, so this text cannot drift from the shipped behaviour.

With `parallel_tape = FALSE` the budget is per fit; with the default `parallel_tape = TRUE` the tapes are built concurrently and the guard multiplies the workload by the realized thread count (see `n_cores` above for how that count is realized).

One unit is one weighted observation-draw. All figures are measured by `inst/dev/tmb_benchmark_m` whose raw results are stored in `inst/extdata/memory_calibration.csv`.

Retained tape size depends on `n * draws` alone: tape (MiB) = 13.374 + 0.001164 * units, with R^2 = 0.9994 (1221 bytes per unit over the largest workloads; per-unit cost is higher at small workloads because of the fixed intercept).

The budget is set on *peak* working set, not on retained tape, because peak is what exhausts memory: TMB records the whole likelihood before pruning it to each parallel region, so peak runs about 6.11 times the retained tape plus first-evaluation growth, or 12083 bytes per unit measured directly against peak. The default of 700,000 units therefore targets a peak of about 8 GiB for one fit.

Families cost different amounts per unit, so each carries a weight – the largest *peak* ratio to Famoye observed at matched workload, rounded up to the next tenth: independence 0.7, famoye 1, frank 3.6, gaussian 0.9, clayton 1.1. Frank peaks at over three and a half times Famoye, so treating it as unweighted would under-budget it more than threefold.

Raise `max_workload` deliberately against the memory you actually have, not to make one particular dataset fit.

As of `rpbnb_tmb_max_workload()`, the value `rpbnb_tmb_control()` actually uses by default is no longer the fixed figure above: it will auto-detect available memory and budget 80% of it, falling back to the fixed 8 GiB figure only when detection is unavailable on the current platform. Call `rpbnb_tmb_max_workload` directly to set your own budget or detection fraction.

`parallel_tape` Construct per-thread TMB tapes concurrently. The default `TRUE` builds them in parallel for faster tape construction; objective and gradient evaluation remains parallel either way. Set `FALSE` to build tapes sequentially instead and reduce peak memory. Concurrent tape construction multiplies the per-tape memory

workload the max_workload/tape_chunks guard sizes against by the realized thread count (see max_workload above); pairing parallel_tape = TRUE with max_workload = Inf disables that guard entirely; rpbnb_control() warns when it sees that combination.

tape_chunks — TMB engine, SML fits only. Number of draw chunks to split draws into (see draws at fit_rpbnb_tmb()). NULL (default) auto-selects the smallest sufficient count when the weighted workload exceeds max_workload, or 1L (no chunking) when it does not. Set explicitly to pin a layout regardless of the auto-threshold; must not exceed draws. Chunking is exact for the requested draws (not an approximation) at the cost of somewhat slower gradient evaluations, and a chunked fit has no taped Hessian – confint(method = "profile")/rpbnb_tmb_dependence_profile() fall back to a Wald interval with a warning; Wald/optimHess inference (the default) is unaffected. Ignored for method = "laplace" (which has no draw dimension to chunk) and by every non-TMB estimator.

## Value

An object of class c("rpbnb_control", "rpbnb_tmb_control") (a named list). It carries both class names so that every historical inherits(control, ...) check in the package and in user code accepts it.

## Which parameters apply to which estimator

| Parameter | fit_bnb() | fit_rpbnb() | fit_rpbnb_tmb() |
|---|---|---|---|
| method, compute_se, hess_eps, hess_r | yes | yes | ignored |
| iterlim, reltol, print_level | yes | yes | yes |
| halton_burn, n_cores | ignored | yes | yes |
| hessian | yes | ignored | ignored |
| se_method | ignored | yes | ignored |
| gradtol, restarts, max_threads, max_workload, parallel_tape | ignored | ignored | yes |
| draws_hessian | ignored | ignored | ignored |

Note that iterlim and n_cores mean different things to different estimators – a maxLik BFGS iteration limit versus an nlminb one, worker *processes* versus OpenMP *threads*. They are not translated; each estimator reads the number and applies its own meaning to it.

## Defaults that depend on the estimator

iterlim and print_level default to NULL, which means "this estimator's own default": iterlim is 300 under maxLik and 500 under nlminb. print_level is 2 under maxLik and, as of 0.4.6, 1 under the TMB engine (it was 0 – fully silent – before that, which left a long TMB fit printing nothing at all while it ran). The TMB default of 1 shows TMB's own progress without nlminb's per-iteration parameter vectors; see print_level below for what each level prints. Supplying either explicitly overrides that for every estimator; print_level = 0 restores silence. max_threads defaults to n_cores and max_workload is computed from available memory by rpbnb_tmb_max_workload() the first time a TMB fit needs it (so a non-TMB fit never pays for the memory probe).

## How n_cores, max_workload, and tape_chunks interact (TMB, method = "sml")

These three only interact through parallel_tape. Before fit_rpbnb_tmb() builds a tape, it estimates a weighted workload: nrow(data) * draws * <family weight> * (realized n_cores if parallel_tape = T

max_workload caps that number, and tape_chunks is what lets a fit exceed it anyway by building several smaller tapes instead of one.

- n_cores sets OpenMP threads for the objective/gradient, which are always evaluated in parallel regardless of anything else here. With the default parallel_tape = TRUE, the *realized* thread count (after max_threads and hardware capping) also multiplies the workload above, because each thread builds its own full tape concurrently – so raising n_cores speeds up evaluation but can push a fit that fit comfortably at one thread over max_workload at eight. Set parallel_tape = FALSE to decouple the two: tapes then build one at a time (peak memory unaffected by n_cores) while evaluation stays fully parallel.
- max_workload (NULL auto-detects available memory; see above) is checked against that workload number. Exceeding it does not always error: under method = "sml" the fit auto-chunks instead (next point). Inf disables the check *and* auto-chunking entirely, so nothing sizes the tape for you – pin tape_chunks yourself if you still want the memory benefit with the guard off.
- tape_chunks (SML fits only; ignored under method = "laplace", whose tape scales with nrow(data) alone, not draws) is what absorbs a workload over budget: NULL auto-selects the smallest chunk count that brings the per-chunk workload back under max_workload; an explicit value pins the layout (validated against both draws and the budget). More chunks trade slower gradient evaluations (each chunk's contribution is recomputed on every outer step) and the loss of a taped Hessian (confint(method = "profile")/rpbnb_tmb_dependence_profile() fall back to a Wald interval) for lower peak memory.

Net effect for a large SML fit: pick n_cores for speed first. If that trips max_workload, either let auto-chunking absorb it, set parallel_tape = FALSE if the extra peak memory isn't worth the tape-build speedup, or raise max_workload deliberately against memory you actually have (see rpbnb_tmb_max_workload()).

## See Also

rpbnb_tmb_max_workload(), rpbnb(), fit_rpbnb(), fit_rpbnb_tmb(), fit_bnb()

## Examples

```
rpbnb_control(method = "BFGS", iterlim = 200)
rpbnb_control(hessian = "analytic")
# The same object drives either engine; the TMB-only knobs are simply
# ignored by the classic one (and reported as ignored in its summary).
rpbnb_control(n_cores = 4, gradtol = 1e-6, se_method = "opg")
```

---

# rpbnb_elasticities *Elasticities and semi-elasticities for a random-parameter bivariate NB model*

---

## Description

Continuous elasticities $x_{ij}\,(\partial\mu_i/\partial x_{ij})/\mu_i$ and binary semi-elasticities $\mu_i(x_j = 1)/\mu_i(x_j = 0) - 1$, built on the Monte-Carlo integrated population mean $\mu_i = E_\beta[\exp(x_i'\beta)]$ of an fit_rpbnb() fit. Under fixed coefficients these reduce to $\beta_j E[x_j]$ and $\exp(\beta_j) - 1$. Standard errors use a numeric delta method over the equation's mean and log-scale parameters.

## Usage

```
rpbnb_elasticities(
  fit,
  which = c("y1", "y2", "both"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE,
  n_cores = 1L,
  scaling = NULL,
  log_vars = NULL
)
```

## Arguments

| | |
|---|---|
| fit | An rpbnb_fit object from fit_rpbnb(). |
| which | Which margin(s): "y1", "y2", or "both". |
| type | "AME" (average over the sample) or "MEM" (evaluated at the mean row). |
| vars | Optional variable names or indices to restrict output. |
| include_intercept | Logical; include the intercept term. |
| digits | Number of decimal places for printed output. |
| print_output | Logical; if FALSE, suppress printing. |
| n_cores | Number of worker processes for the delta-method standard-error jacobian (1 = sequential, the default). When n_cores > 1, the jacobian's independent per-parameter columns are dispatched across a parallel::makeCluster() cluster (one cluster per call, shared across every equation which computes); results are numerically identical to the sequential path. Falls back to sequential with a warning if the parallel package is unavailable. |
| scaling | Optional named list of c(center =, scale =) pairs, one per covariate that was centred and/or scaled before fitting (e.g. via rpbnb(standardize = TRUE), whose $scaling field is exactly this shape), used to report the results in the covariate's original units. See rpbnb_marginal_effects()'s scaling documentation for the full explanation (elasticities and marginal effects share it verbatim). |
| log_vars | Optional character vector naming covariates that are ALREADY a log; see rpbnb_marginal_effects()'s log_vars documentation. |

## Value

A data frame (single margin, invisibly) or a named list of data frames (both), each with columns Name, Estimate, StdErr, z, p, Signif, var_type.

## See Also

bnb_elasticities() for fixed-coefficient bnb_fit models.

## Examples

```
sim <- simulate_rpbnb(n = 400,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 100)
rpbnb_elasticities(fit, which = "both", type = "AME")
```

---

`rpbnb_marginal_effects` *Marginal effects for a random-parameter bivariate NB model*

---

## Description

Average marginal effects (AME) or marginal effects at the mean (MEM) for each margin of an `fit_rpbnb()` fit, built on the Monte-Carlo integrated population mean $\mu_i = E_\beta[\exp(x_i'\beta)]$ (the same estimand as `predict.rpbnb_fit()`, reusing the fit's stored draws). Continuous effects are $\partial\mu_i/\partial x_{ij} = \text{mean}_r\, \text{coef}_{rj} \exp(\text{lp}_{ir})$ (the realized coefficient per draw, so random and fixed columns are handled uniformly); binary (0/1) effects are the integrated discrete difference $E[Y|x_j = 1] - E[Y|x_j = 0]$. Standard errors use a numeric delta method over the equation's mean and log-scale parameters.

## Usage

```
rpbnb_marginal_effects(
  fit,
  which = c("y1", "y2", "both", "all"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4,
  print_output = TRUE,
  n_cores = 1L,
  scaling = NULL,
  log_vars = NULL
)
```

## Arguments

| | |
|---|---|
| `fit` | An `rpbnb_fit` object from `fit_rpbnb()`. |
| `which` | Which margin(s): "y1", "y2", "both", or "all". |
| `type` | "AME" (average over the sample) or "MEM" (effect at the mean row). |
| `vars` | Optional variable names or indices to restrict output. |
| `include_intercept` | Logical; include the intercept term. |
| `digits` | Number of decimal places for printed output. |
| `print_output` | Logical; if `FALSE`, suppress printing. |

`n_cores` Number of worker processes for the delta-method standard-error jacobian (1 = sequential, the default). When `n_cores` > 1, the jacobian's independent per-parameter columns are dispatched across a `parallel::makeCluster()` cluster (one cluster per call, shared across every equation `which` computes); results are numerically identical to the sequential path. Falls back to sequential with a warning if the `parallel` package is unavailable.

`scaling` Optional named list of `c(center =, scale =)` pairs, one per covariate that was centred and/or scaled before fitting (e.g. via `rpbnb(standardize = TRUE)`, whose `$scaling` field is exactly this shape), used to report the results in the covariate's original units. Columns not named are left alone, so binary indicators and untransformed variables need no entry.

Only the reported quantity is rescaled; the fitted design is not rebuilt. That distinction is not cosmetic when a standardized covariate also carries a random coefficient: the random term enters as `x_std * dev`, so substituting `x_raw = x_std * s + c` would add a `(c/s) * dev` random intercept the model never estimated – the disguised random intercept centring was meant to remove, back again. The chain rule avoids that: `d(mu)/d(x_raw) = (d(mu)/d(x_std))/s`, and the elasticity's leading factor becomes `x_raw/s = x_std + c/s`. Binary effects are untouched.

`log_vars` Optional character vector naming covariates that are ALREADY a log, so results are reported per unit of the underlying variable `v_j = exp(x_j)` rather than per unit of `x_j` itself. Without this the elasticity formula treats `log v` as an ordinary regressor and returns `xbar * b`, the elasticity with respect to the LOG – for a log-traffic column that can be an order of magnitude off from the elasticity with respect to traffic itself (the coefficient). Rejects binary columns (a 0/1 indicator is not a log-transformed variable).

## Details

For a model fitted with an `offset()`, absolute marginal effects scale with the offset-inclusive mean: AME uses each observation's training offset, and MEM uses the mean training offset.

## Value

A data frame (single margin, invisibly) or a named list of data frames (`both`/`all`), each with columns `Name`, `Estimate`, `StdErr`, `z`, `p`, `Signif`, `var_type`.

## See Also

`bnb_marginal_effects()` for fixed-coefficient `bnb_fit` models.

## Examples

```
sim <- simulate_rpbnb(n = 400,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                 draws = 100)
rpbnb_marginal_effects(fit, which = "y1", type = "AME")
```

## rpbnb_threads *Number of CPU threads available for the multithreaded likelihood*

### Description

Number of CPU threads available for the multithreaded likelihood

### Usage

```
rpbnb_threads()
```

### Value

Integer thread count reported by OpenMP (1 if OpenMP is unavailable).

## rpbnb_tmb_boundary_tests *Boundary-corrected LR tests for an rpbnb_tmb_fit's boundary parameters*

### Description

Runs a likelihood-ratio test for every random-coefficient scale (`sd1:*`, `sd2:*`) and NB2 dispersion (`m1`, `m2`) of a fitted TMB-engine random-parameter bivariate NB model, and merges them into one table. These are the parameters whose null lies on the boundary of the parameter space (scale = 0, or dispersion `m` = `0` = Poisson), for which `summary(fit)` reports no Wald z/p; each test uses the 50:50 chi-square boundary correction of `lr_test()` (`boundary = TRUE`).

### Usage

```
rpbnb_tmb_boundary_tests(
  fit,
  data,
  control = NULL,
  which = c("sd", "dispersion"),
  draws = fit$draws,
  disable_parallel_gaussian = FALSE,
  force_parallel_gaussian = NULL,
  sml_fallback = TRUE
)
```

### Arguments

| | |
|---|---|
| `fit` | A converged `rpbnb_tmb_fit` (from `fit_rpbnb_tmb()` or `rpbnb(engine = "tmb")`). |
| `data` | The data frame the model was fit on. Required – the fit object does not store it, and every restricted model is refit on the same data. With `standardize = TRUE`, pass the standardized data (`rpbnb(engine = "tmb", boundary_tests = TRUE)` does this automatically; a manual call needs `rpbnb:::.apply_scaling(data, fit$scaling)`). |

`control` An `rpbnb_tmb_control()` for the restricted refits. Defaults to `rpbnb_tmb_control(print_level =` – the same `n_cores` the original fit was called with (1 if `fit` predates the stored `$parallel` field). `max_workload` propagates `fit`'s own value when `fit` draw-chunked (see `draws` at `fit_rpbnb_tmb()`), so a restricted refit re-chunks to stay in budget instead of rebuilding one full tape; both stay at today's pre-chunking defaults (`Inf`, `NULL`) when `fit` did not chunk (or predates `$tape_integration`). When this `draws` equals `fit$draws` (the default), `tape_chunks` also propagates `fit`'s own resolved chunk count – needed because a PINNED `tape_chunks` can coexist with `max_workload = Inf` (propagating budget alone would then give the refit no way to reconstruct that layout, and it would silently build one full tape). A `draws` that differs from `fit$draws` gets a fresh layout resolved from the propagated `max_workload` alone, never a stale chunk count carried over from a different draw request. `seed`/`method` are taken from `fit`, not `control`.

`which` Which parameter groups to test, any subset of `"sd"` (the random-coefficient scales), `"dispersion"` (the NB2 dispersions `m1`, `m2`), and `"dependence"` (the association parameter). The default is `c("sd", "dispersion")` – the two boundary-null groups. `"dependence"` is opt-in because it costs another full refit and, for three of the four families, tests an *interior* null whose ordinary Wald `z` in `summary()` is already valid.

The dependence test refits the model with `dependence = "independence"` – the template's own family, which maps `z_dep` out of the free parameters, giving an exact 1-df restriction on the same draws – and reports one row labelled `lam` (Famoye), `theta` (Frank / Kimeldorf), or `rho` (Gaussian). Only Kimeldorf's null (`theta > 0`, so `theta = 0` is a boundary) takes the 50:50 mixture; Famoye, Frank, and Gaussian nulls are interior and get an ordinary chi-square(1).

`draws` Number of Halton simulation draws for the restricted refits. Defaults to `fit$draws` – the same number `fit` itself used. Raising this trades refit cost for precision without needing to refit `fit` at a higher `draws`; lowering it is cheaper but noisier. A value other than `fit$draws` still shares `fit$seed` (so the Halton sequence's *prefix* is identical) but no longer draws the exact same simulated log-likelihood surface as `fit`, so the LR statistic picks up simulation noise beyond the restriction under test – prefer the default unless you have a specific reason to diverge.

`disable_parallel_gaussian`
Opt-out that restricts each restricted refit's Gaussian-copula evaluation to one thread (see `?fit_rpbnb_tmb`), forwarded to every refit. Default `FALSE` (multi-threaded, the 0.4.6+ default). This is intentionally a separate argument rather than something read off `fit`: `fit` does not record whether the original fit opted out, so a caller who wants single-threaded refits must pass `disable_parallel_gaussian = TRUE` here even when the original `fit_rpbnb_tmb()`/`rpbnb()` call also did.

`force_parallel_gaussian`
Deprecated (pre-0.4.6 polarity); use `disable_parallel_gaussian` instead. `TRUE` (the old opt-in) is a no-op beyond its deprecation warning; an explicit `FALSE` is honored as `disable_parallel_gaussian = TRUE`.

`sml_fallback` When `fit` was estimated by `method = "laplace"` and a restricted refit's Laplace pair cannot be trusted, re-run **that one test's** LR with both sides estimated by `method = "sml"` instead of reporting `NA` (or a clamped 0). Default `TRUE`. Two triggers: the restricted refit **fails to converge**, or it reports convergence with a restricted logLik *above* the full fit's (a negative raw LR). The second is not a rounding curiosity: the models are nested, so at true optima the restricted logLik can never exceed the full fit's – a negative LR proves at least one Laplace value is wrong, either a full fit that stopped short of its optimum (observed at -0.002 to

-1.2 nats: the warm-started restricted refit out-polished it) or a restricted fit that climbed a spurious ridge of the approximation itself (observed at -3838 nats: near a singular inner Hessian the Laplace log-likelihood rises without bound, and nlminb reports code 0 there). Clamping such a statistic to 0 would silently turn a real effect into "no evidence".

This exists because some restrictions leave Laplace no valid optimum to converge to. Pinning a margin to Poisson can push the dependence strong enough (observed on the truck data's `m1` test under a Frank copula: theta driven to about 20, Kendall's tau 0.8) that the per-observation cell probability is non-log-concave in the random effects – and the Laplace approximation differentiates through an inner Newton that requires exactly the log-concavity the restricted model no longer has. No inner-solver setting fixes that (`tol10 = 0` clears TMB's `"Newton drop out"` but the outer fit still ends at nlminb `false convergence (8)` with a gradient of 1e10); SML has no inner Newton and is not exposed to it.

The fallback refits the **full model too** under SML (once, cached across tests) at these same `draws` and `fit$seed`, and takes the LR between the two SML fits – never between a Laplace logLik and an SML logLik, which are different approximations of the likelihood and whose difference is not an LR statistic. Rows that used the fallback are listed in the result's `sml_fallback` attribute and announced by a `message()` (silenced by `control$print_level = 0` like the other announcements). If the SML pair fails to converge as well, the row is `NA` with a warning, exactly as before. Ignored when `fit` itself was estimated by SML (there is nothing different to fall back to).

## Details

Each test refits a properly nested restricted model, otherwise identical to `fit` (same formulas, random-coefficient specification, dependence, seed, and estimator, and by default the same `draws`), warm-started from `fit$coef` and run with `inference = "none"` since the LR test needs only `logLik` and the parameter count.

**Dispersions** are restricted with `poisson_1`/`poisson_2 = TRUE`, the template's exact `m = 0` branch.

**Scales** are restricted by pinning that coefficient's `log_sd` at the parameterization's zero (`-20`; the template clamps `log_sd` to `[-20, 20]` and computes `sd = exp(log_sd)`, so this is `sd = 2.1e-9` – numerically zero on every draw) and holding it out of the free-parameter count, giving a 1-df restriction. With **multiple random coefficients in an equation, each scale is tested individually.**

Pinning the scale rather than dropping the coefficient from `random_1`/`random_2` is what preserves **common random numbers**: the Halton draw matrix keeps the same width, so every *other* random coefficient draws from exactly the dimensions it did in the full fit, and the two simulated log-likelihoods differ only by the restriction under test. Dropping the name instead would renumber the remaining coefficients' Halton dimensions, and the LR statistic would absorb that reshuffling as extra simulation noise.

A `message()` (`"Boundary LR test: <parameter>..."`) reports each restricted refit right before it starts, unless `control$print_level` is `0`; suppress it with `suppressMessages()` if needed.

## Value

An object of class `rpbnb_boundary_tests` – the same class `rpbnb_boundary_tests()` returns, with columns `Parameter`, `LR`, `df`, `p.value`, `Signif` (one row per boundary parameter) and the same `print()` method – so the two engines' results are interchangeable wherever that class is consumed (e.g. `summary.rpbnb_tmb_fit()`'s scale and dispersion blocks, once `$boundary_tests` is attached to the fit).

Scale rows are labelled by the distribution's own scale parameter (`sd` for normal/lognormal, `w` for uniform/triangular half-width, `s` for a lognormal log-scale), matching `summary()`'s row names.

The `sml_fallback` attribute is a character vector of the `Parameter` rows whose LR came from the SML fallback pair (see the `sml_fallback` argument); `character(0)` when every test ran under `fit`'s own estimator.

## See Also

`lr_test()`, `fit_rpbnb_tmb()`, `rpbnb_boundary_tests()` (the classic-engine counterpart)

## Examples

```
sim <- simulate_rpbnb(n = 600,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
fit <- fit_rpbnb_tmb(y1 ~ x1, y2 ~ x1, data = sim$data, random_1 = "x1",
                     draws = 100)
rpbnb_tmb_boundary_tests(fit, sim$data)
```

---

# rpbnb_tmb_control *Control parameters for the TMB engine (alias of* `rpbnb_control()`*)*

---

## Description

Retained so that code written against the pre-unification API keeps working. The two control objects were merged in 0.4.1: this function forwards to `rpbnb_control()` and returns exactly the same object, which every estimator in the package accepts. New code should call `rpbnb_control()` directly.

## Usage

```
rpbnb_tmb_control(
  iterlim = NULL,
  reltol = 1e-08,
  gradtol = 1e-05,
  restarts = 10L,
  print_level = NULL,
  n_cores = 1L,
  max_threads = NULL,
  max_workload = NULL,
  parallel_tape = TRUE,
  halton_burn = 300L,
  tape_chunks = NULL
)
```

## Arguments

| | |
|---|---|
| iterlim | Maximum optimizer iterations. NULL (default) uses 300 for the maxLik fitters and 500 for the TMB engine's nlminb. |
| reltol | Relative convergence tolerance. |
| gradtol | Stationarity tolerance for the TMB engine's score, applied *relative* to the objective: the fit is treated as stationary once max(abs(gradient)) falls below gradtol * max(1, abs(nll)). nlminb() declares convergence from its relative function test, which can fire while the score is still far from zero; a Hessian taken there is not a curvature and its inverse can carry negative variances. The fit is restarted until the gradient clears this tolerance; missing it is not warned about on its own, because the copula families legitimately optimise against a clamp or a probability floor where no step improves the objective, but it is reported in the warning raised when the Hessian actually fails to factor, and is kept on the fit as optimizer$max_abs_gradient. The scaling matters because the score is a sum over observations, so an absolute cutoff would tighten with sample size for no statistical reason. |
| restarts | Maximum number of times to restart nlminb() from its own answer while gradtol is unmet. Restarting resets the trust region and step scaling that stalled the first solve. Set to 0L for the single-call behaviour of earlier versions. |
| print_level | Optimizer verbosity, and the switch that silences the boundary-test progress messages. NULL (default) uses 2 under maxLik and 1 under the TMB engine. Under TMB it drives two separate switches – MakeADFun(silent = print_level == 0) and nlminb's trace = max(0, print_level - 1) – so the levels are:<br>0 Silent (the pre-0.4.6 TMB default).<br>1 TMB's own output only: an outer mgc: line per outer evaluation, plus the one-time tape/atomic construction on the first fit of a session. No nlminb trace.<br>2 Adds nlminb's per-iteration objective and parameter vector – the lowest level that traces every iteration.<br>>2 nlminb's trace is a print *interval*, not a verbosity level, so higher values print the objective *less* often.<br>Note that rpbnb_tmb_boundary_tests() builds its own control when one is not supplied, so its restricted refits print at their own default regardless of this setting. |
| n_cores | Worker processes for fit_rpbnb()'s optional cluster path, or OpenMP threads for fit_rpbnb_tmb() (1 = sequential in both cases). Under the TMB engine the *realized* thread count – after max_threads and hardware capping – is also what max_workload/tape_chunks size the tape against when parallel_tape = TRUE; see "How n_cores, max_workload, and tape_chunks interact" below. |
| max_threads | Maximum OpenMP threads permitted for one TMB fit. NULL (default) means n_cores, so threads are not capped unless set explicitly below n_cores. |
| max_workload | Maximum weighted observation-draw evaluations permitted before TMB tape construction; Inf disables the guard. The default and every figure here are derived from TAPE_CALIBRATION, so this text cannot drift from the shipped behaviour.<br>With parallel_tape = FALSE the budget is per fit; with the default parallel_tape = TRUE the tapes are built concurrently and the guard multiplies the workload by the realized thread count (see n_cores above for how that count is realized). |

One unit is one weighted observation-draw. All figures are measured by `inst/dev/tmb_benchmark_m` whose raw results are stored in `inst/extdata/memory_calibration.csv`.

Retained tape size depends on `n * draws` alone: tape (MiB) = 13.374 + 0.001164 * units, with R^2 = 0.9994 (1221 bytes per unit over the largest workloads; per-unit cost is higher at small workloads because of the fixed intercept).

The budget is set on *peak* working set, not on retained tape, because peak is what exhausts memory: TMB records the whole likelihood before pruning it to each parallel region, so peak runs about 6.11 times the retained tape plus first-evaluation growth, or 12083 bytes per unit measured directly against peak. The default of 700,000 units therefore targets a peak of about 8 GiB for one fit.

Families cost different amounts per unit, so each carries a weight – the largest *peak* ratio to Famoye observed at matched workload, rounded up to the next tenth: independence 0.7, famoye 1, frank 3.6, gaussian 0.9, clayton 1.1. Frank peaks at over three and a half times Famoye, so treating it as unweighted would under-budget it more than threefold.

Raise `max_workload` deliberately against the memory you actually have, not to make one particular dataset fit.

As of `rpbnb_tmb_max_workload()`, the value `rpbnb_tmb_control()` actually uses by default is no longer the fixed figure above: it will auto-detect available memory and budget 80% of it, falling back to the fixed 8 GiB figure only when detection is unavailable on the current platform. Call `rpbnb_tmb_max_workload` directly to set your own budget or detection fraction.

`parallel_tape` Construct per-thread TMB tapes concurrently. The default `TRUE` builds them in parallel for faster tape construction; objective and gradient evaluation remains parallel either way. Set `FALSE` to build tapes sequentially instead and reduce peak memory. Concurrent tape construction multiplies the per-tape memory workload the `max_workload`/`tape_chunks` guard sizes against by the realized thread count (see `max_workload` above); pairing `parallel_tape = TRUE` with `max_workload = Inf` disables that guard entirely; `rpbnb_control()` warns when it sees that combination.

`halton_burn` Number of leading Halton points discarded before forming the simulation draws.

`tape_chunks` TMB engine, SML fits only. Number of draw chunks to split `draws` into (see `draws` at `fit_rpbnb_tmb()`). `NULL` (default) auto-selects the smallest sufficient count when the weighted workload exceeds `max_workload`, or `1L` (no chunking) when it does not. Set explicitly to pin a layout regardless of the auto-threshold; must not exceed `draws`. Chunking is exact for the requested draws (not an approximation) at the cost of somewhat slower gradient evaluations, and a chunked fit has no taped Hessian – `confint(method = "profile")`/`rpbnb_tmb_dependence_profile()` fall back to a Wald interval with a warning; Wald/optimHess inference (the default) is unaffected. Ignored for `method = "laplace"` (which has no draw dimension to chunk) and by every non-TMB estimator.

## Details

Only the arguments you actually supply are forwarded, so an untouched `iterlim`/`print_level` still resolves to the TMB engine's own defaults (500 and 1) when the object is used for a TMB fit – and to the `maxLik` defaults if the same object is handed to `fit_rpbnb()`.

## Value

The `rpbnb_control()` object.

## See Also

rpbnb_control()

## Examples

```
identical(unclass(rpbnb_tmb_control(n_cores = 2)),
          unclass(rpbnb_control(n_cores = 2)))
```

---

`rpbnb_tmb_dependence_profile` *Confidence interval for a fitted dependence parameter*

---

## Description

Returns a profile-likelihood interval for the dependence parameter of a fitted model, computed on the unconstrained working scale and mapped through the family's monotone link. This is the tool to reach for when `summary()` reports `NA` for the dependence standard error: at a boundary the delta-method derivative collapses, so $SE = |d\theta/dz| \cdot SE(z)$ is a $0 \times \infty$ product and no symmetric standard error exists. A profile interval needs no derivative.

## Usage

```
rpbnb_tmb_dependence_profile(
  fit,
  level = 0.95,
  method = c("profile", "wald"),
  ...
)
```

## Arguments

| | |
|---|---|
| `fit` | An object of class `rpbnb_tmb_fit`. |
| `level` | Coverage level; one number strictly between 0 and 1. |
| `method` | `"profile"` (default) for a profile-likelihood interval, or `"wald"` for a Wald interval on the working scale mapped through the same link. `"profile"` requires the TMB objective, retained only under `keep = "full"`; when it is unavailable this degrades to `"wald"` with a warning rather than failing. |
| `...` | Passed to `tmbprofile`, e.g. `ytol` or `parm.range`. `lincomb` and `slice` are not supported and raise an error: the first would profile a different quantity than `z_dep` while still being reported as if it were, and the second returns a likelihood slice rather than a profile. |

## Details

For Famoye this is the usual situation, because the admissible lambda interval is frozen at the starting values (see `lambda_bounds` in `fit_rpbnb_tmb`). An interval around a pinned estimate is still an interval around an artefact – widen the box first by refitting from better starting values.

## Value

A data frame with one row per reported dependence quantity and columns `parameter`, `estimate`, `lower`, `upper`, `level`, and `method` – the method actually used, which may differ from the one requested. The raw profile, when one was computed, is attached as `attr(, "profile")` so it can be plotted.

## See Also

`fit_rpbnb_tmb`

## Examples

```
## Not run:
fit <- fit_rpbnb_tmb(y1 ~ x, y2 ~ x, data = d,
                     dependence = "famoye", keep = "full")
rpbnb_tmb_dependence_profile(fit)
plot(attr(rpbnb_tmb_dependence_profile(fit), "profile"))

## End(Not run)
```

---

# rpbnb_tmb_elasticities

*Elasticities for a rpbnb_tmb model*

---

## Description

Continuous elasticities $x_j/E[Y] \cdot \partial E[Y]/\partial x_j$ and binary semi-elasticities $E[Y|x_j = 1]/E[Y|x_j = 0] - 1$. Built on the same Monte-Carlo integrated mean used by `rpbnb_tmb_marginal_effects`.

## Usage

```
rpbnb_tmb_elasticities(
  fit,
  which = c("y1", "y2", "both"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4L,
  scaling = NULL,
  log_vars = NULL,
  ...
)
```

## Arguments

| | |
|---|---|
| `fit` | An object of class `rpbnb_tmb_fit`. |
| `which` | Which margin: `"y1"`, `"y2"`, or `"both"`. |
| `type` | `"AME"` (average over the sample) or `"MEM"` (at the sample mean of covariates). |
| `vars` | Optional character vector of variable names to restrict output. |
| `include_intercept` | Logical; include the intercept term in output. |

`digits` Number of decimal places for printed table entries.

`scaling` Optional named list of `c(center =, scale =)` pairs, one per covariate that was centred and/or scaled before fitting, used to report the results in the covariate's original units. Columns not named are left alone, so binary indicators and untransformed variables need no entry.

Only the reported quantity is rescaled; the fitted design is not rebuilt. That distinction is not cosmetic when a standardized covariate also carries a random coefficient. The random term enters as `x_std * dev`, so substituting `x_raw = x_std * s + c` would add a `(c/s) * dev` random intercept the model never estimated – which is how a centred random slope turns back into the disguised random intercept that centring was meant to remove. The chain rule avoids that entirely: $\partial\mu/\partial x_{raw} = (\partial\mu/\partial x_{std})/s$, and the elasticity's leading factor becomes $x_{raw}/s = x_{std} + c/s$. Binary effects are untouched.

`log_vars` Optional character vector naming covariates that are ALREADY a log, so that results are reported per unit of the underlying variable $v_j = \exp(x_j)$ rather than per unit of $x_j$ itself.

Without this the elasticity formula treats $\log v$ as an ordinary regressor and returns $\bar{x}b$, the elasticity with respect to the LOG – for the truck model's `LNAADT_3` that is 8.59, where the elasticity with respect to AADT is the coefficient itself, 0.871. A ten-fold overstatement that looks perfectly plausible in a table.

Because $\partial \log\mu/\partial \log v = (\partial\mu/\partial x)/\mu$, the elasticity is $E[d\mu/\mu]/s$ and the marginal effect is $E[d\mu/(sv)]$, with $v = \exp(x_{std}s+c)$ picking up any `scaling` applied to the logged column. Named columns are reported with type `"log-continuous"`. Binary columns are rejected.

`...` Not used.

## Value

A data frame or named list of data frames.

---

`rpbnb_tmb_marginal_effects`
*Marginal effects for a rpbnb_tmb model*

---

## Description

Computes average marginal effects (AME) or marginal effects at the mean (MEM) for a fitted random-parameter bivariate negative binomial model. For continuous covariates the effect is $\partial E[Y]/\partial x_j$; for binary covariates it is the discrete difference $E[Y|x_j = 1] - E[Y|x_j = 0]$. When a coefficient is random the per-draw realized coefficients are used to form the Monte-Carlo integrated mean.

## Usage

```
rpbnb_tmb_marginal_effects(
  fit,
  which = c("y1", "y2", "both"),
  type = c("AME", "MEM"),
  vars = NULL,
  include_intercept = FALSE,
  digits = 4L,
```

```
  scaling = NULL,
  log_vars = NULL,
  ...
)
```

## Arguments

| | |
|---|---|
| fit | An object of class rpbnb_tmb_fit. |
| which | Which margin: "y1", "y2", or "both". |
| type | "AME" (average over the sample) or "MEM" (at the sample mean of covariates). |
| vars | Optional character vector of variable names to restrict output. |
| include_intercept | Logical; include the intercept term in output. |
| digits | Number of decimal places for printed table entries. |
| scaling | Optional named list of c(center =, scale =) pairs, one per covariate that was centred and/or scaled before fitting, used to report the results in the covariate's original units. Columns not named are left alone, so binary indicators and untransformed variables need no entry.<br>Only the reported quantity is rescaled; the fitted design is not rebuilt. That distinction is not cosmetic when a standardized covariate also carries a random coefficient. The random term enters as x_std * dev, so substituting x_raw = x_std * s + c would add a (c/s) * dev random intercept the model never estimated – which is how a centred random slope turns back into the disguised random intercept that centring was meant to remove. The chain rule avoids that entirely: $\partial\mu/\partial x_{raw} = (\partial\mu/\partial x_{std})/s$, and the elasticity's leading factor becomes $x_{raw}/s = x_{std} + c/s$. Binary effects are untouched. |
| log_vars | Optional character vector naming covariates that are ALREADY a log, so that results are reported per unit of the underlying variable $v_j = \exp(x_j)$ rather than per unit of $x_j$ itself.<br>Without this the elasticity formula treats $\log v$ as an ordinary regressor and returns $\bar{x}b$, the elasticity with respect to the LOG – for the truck model's LNAADT_3 that is 8.59, where the elasticity with respect to AADT is the coefficient itself, 0.871. A ten-fold overstatement that looks perfectly plausible in a table.<br>Because $\partial \log\mu/\partial \log v = (\partial\mu/\partial x)/\mu$, the elasticity is $E[d\mu/\mu]/s$ and the marginal effect is $E[d\mu/(sv)]$, with $v = \exp(x_{std}s+c)$ picking up any scaling applied to the logged column. Named columns are reported with type "log-continuous". Binary columns are rejected. |
| ... | Not used. |

## Value

A data frame (single margin) or named list of two data frames ("both").

rpbnb_tmb_max_workload *Compute a TMB workload budget from a memory figure*

## Description

Companion to `rpbnb_tmb_control()`'s `max_workload`: rather than picking a weighted-observation-draw count directly, state a memory budget and let this function do the arithmetic `TAPE_CALIBRATION` implies.

## Usage

```
rpbnb_tmb_max_workload(budget_gib = NULL, fraction = 0.8)
```

## Arguments

| | |
|---|---|
| `budget_gib` | Optional memory budget in GiB, stated explicitly. When supplied, used as-is – `fraction` does not apply, because a number you state yourself is not second-guessed with a discount. When omitted (the default), available memory is auto-detected and `fraction` of it is budgeted instead. |
| `fraction` | Of auto-detected available memory, the fraction to actually budget; one number in `(0, 1]`. Available memory fluctuates and competes with other processes, so budgeting all of it risks the guard passing a fit that then exhausts memory anyway. Ignored when `budget_gib` is supplied. |

## Details

With `budget_gib` omitted, this is also `rpbnb_tmb_control()`'s own default: every fit that doesn't set `max_workload` explicitly already goes through this function.

## Value

One positive numeric workload value, on the same scale as `rpbnb_tmb_control()`'s `max_workload`.

## Examples

```
rpbnb_tmb_max_workload(budget_gib = 16)
## Not run:
ctrl <- rpbnb_tmb_control(max_workload = rpbnb_tmb_max_workload())

## End(Not run)
```

`simulate_bnb` *Simulate data from the Famoye/Sarmanov bivariate NB distribution*

## Description

Generates paired count outcomes (`y1, y2`) from the fixed-parameter Famoye/Sarmanov bivariate NB2 joint PMF:

## Usage

```
simulate_bnb(
  n,
  beta1,
  beta2,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  lambda = 0,
  covariates = NULL,
  seed = NULL
)
```

## Arguments

| | |
|---|---|
| `n` | Number of observations. |
| `beta1, beta2` | Named numeric vectors of coefficients for each equation; must include `"(Intercept)"`. |
| `dispersion` | Named numeric `c(m1 = ..., m2 = ...)` NB2 dispersion parameters (variance = mu + m * mu^2). |
| `lambda` | Famoye/Sarmanov dependence parameter. Must lie within the valid bounds implied by the marginal means and dispersions. |
| `covariates` | Optional data frame of covariates. If `NULL`, standard- normal columns are generated for every non-intercept coefficient name. |
| `seed` | Optional random seed. If `NULL` (default) the RNG is left untouched and draws continue from the caller's current stream, so repeated calls yield distinct datasets; supply an integer for reproducible output. |

## Details

$$P(Y_1 = y_1, Y_2 = y_2) = p_1(y_1)\, p_2(y_2)\, [1 + \lambda(e^{-y_1} - c_1)(e^{-y_2} - c_2)]$$

where $c_k = E[e^{-Y_k}]$ under NB2($\mu_k, m_k$).

## Value

A list with:

`data` data frame with `y1`, `y2`, and covariate columns

`mu` data frame with `mu1`, `mu2` (per-obs conditional means)

`true` list of true parameters: `beta1`, `beta2`, `dispersion`, `lambda`

`settings` list with `n` and `seed`

`meta` list with `seed` and `r_version`

## Examples

```
sim <- simulate_bnb(n = 500,
  beta1 = c("(Intercept)" = 0.5, x1 = 0.3),
  beta2 = c("(Intercept)" = 0.2, x1 = -0.2),
  dispersion = c(m1 = 0.4, m2 = 0.5), lambda = 0.1, seed = 1)
head(sim$data)
```

---

`simulate_rpbnb` *Simulate data from a random-parameter bivariate NB process*

---

## Description

Simulate data from a random-parameter bivariate NB process

## Usage

```
simulate_rpbnb(
  n,
  beta1,
  beta2,
  random_1 = NULL,
  random_2 = NULL,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  lambda = 0,
  covariates = NULL,
  seed = NULL
)
```

## Arguments

| | |
|---|---|
| `n` | Number of observations. |
| `beta1`, `beta2` | Named numeric vectors of fixed coefficient means per equation; must include "(Intercept)". |
| `random_1`, `random_2` | Named lists giving random coefficients. Each value is a list with `dist` (one of "normal", "lognormal", "uniform", "triangular"; default "normal"), `scale` (or `sd`) for the dispersion, and `sign` (-1/1, lognormal only). Means come from `beta1`/`beta2`; for a lognormal coefficient the `beta` entry is the log-location and the realized coefficient is `sign * exp(log_location + scale * z)`. |
| `dispersion` | Named numeric `c(m1 = ..., m2 = ...)` NB2 dispersions. |
| `lambda` | Famoye dependence parameter (0 = independent margins). |
| `covariates` | Optional data frame of covariates; if NULL, standard-normal columns are generated for every non-intercept name. |
| `seed` | Optional random seed. If `NULL` (default) the RNG is left untouched and draws continue from the caller's current stream, so repeated calls yield distinct datasets; supply an integer for reproducible output. |

## Value

A list with `data` (y1, y2, covariates), `coef_realized` (per-obs coefficients per equation), `mu` (conditional means), `true` (parameters), `settings`, and `meta` (R/seed/timestamp passed by caller).

## Examples

```
sim <- simulate_rpbnb(n = 500,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  random_1 = list(x1 = list(sd = 0.5)),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
head(sim$data)
```

---

# simulate_rpbnb_copula *Simulate data from a copula RP-BNB process*

---

## Description

Simulate data from a copula RP-BNB process

## Usage

```
simulate_rpbnb_copula(
  n,
  beta1,
  beta2,
  random_1 = NULL,
  random_2 = NULL,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  copula,
  covariates = NULL,
  seed = NULL
)
```

## Arguments

| | |
|---|---|
| n | Number of observations. |
| beta1, beta2 | Named coefficient means; must include "(Intercept)". |
| random_1, random_2 | Random-coefficient specs (see simulate_rpbnb()). |
| dispersion | Named c(m1=, m2=) NB2 dispersions. |
| copula | An copula() object giving the family and native parameter par. |
| covariates | Optional covariate data frame; NULL -> standard-normal columns. |
| seed | Optional RNG seed. |

## Value

list(data, mu, true, settings).

`simulate_rpbnb_tmb` *Simulate data from a bivariate NB process*

## Description

Generates count data from a bivariate negative binomial model with random coefficients and Famoye/Sarmanov or copula dependence.

## Usage

```
simulate_rpbnb_tmb(
  n,
  beta1,
  beta2,
  random_1 = NULL,
  random_2 = NULL,
  dispersion = c(m1 = 0.5, m2 = 0.5),
  dependence = "famoye",
  lambda = 0,
  covariates = NULL,
  seed = NULL
)
```

## Arguments

| | |
|---|---|
| `n` | Number of observations. |
| `beta1`, `beta2` | Named coefficient vectors (must include "(Intercept)"). |
| `random_1`, `random_2` | Random coefficient specs (same format as fit_rpbnb_tmb). |
| `dispersion` | Named vector c(m1 = , m2 = ) of NB2 dispersions. |
| `dependence` | "famoye", "independence", or a copula() object. |
| `lambda` | Famoye dependence parameter (0 = independence). |
| `covariates` | Optional data frame. If NULL, standard-normal covariates generated. |
| `seed` | Optional integer seed. |

## Value

List with $data (data.frame), $truth (parameters), $settings.

## Examples

```
sim <- simulate_rpbnb_tmb(n = 200,
  beta1 = c("(Intercept)" = 0.2, x1 = 0.4),
  beta2 = c("(Intercept)" = 0.1, x1 = -0.3),
  dispersion = c(m1 = 0.4, m2 = 0.5), seed = 1)
head(sim$data)
```

summary.rpbnb_tmb_fit *Summarize a fitted TMB-engine rpbnb model*

## Description

Summarize a fitted TMB-engine rpbnb model

## Usage

```
## S3 method for class 'rpbnb_tmb_fit'
summary(object, digits = 4L, ...)
```

## Arguments

| | |
|---|---|
| object | A fitted model object of class rpbnb_tmb_fit. |
| digits | Number of decimal places for numeric columns in coefficient tables. Default 4. Use a negative value (e.g. -1) for full precision. |
| ... | Not used. |

## Value

The fitted object, invisibly.

# Index